

Efficient and Secure Authenticated Key Exchange Using Weak Passwords

JONATHAN KATZ

University of Maryland

RAFAIL OSTROVSKY

University of California, Los Angeles

AND

MOTI YUNG

Google, Inc. and Columbia University

Abstract. Mutual authentication and authenticated key exchange are fundamental techniques for enabling secure communication over public, insecure networks. It is well known how to design secure protocols for achieving these goals when parties share *high-entropy* cryptographic keys in advance of the authentication stage. Unfortunately, it is much more common for users to share weak, *low-entropy* passwords which furthermore may be chosen from a known space of possibilities (say, a dictionary of English words). In this case, the problem becomes much more difficult as one must ensure that protocols are immune to *off-line dictionary attacks* in which an adversary exhaustively enumerates all possible passwords in an attempt to determine the correct one.

We propose a 3-round protocol for password-only authenticated key exchange, and provide a rigorous proof of security for our protocol based on the decisional Diffie-Hellman assumption. The protocol assumes only public parameters—specifically, a “common reference string”—which can be “hard-coded” into an implementation of the protocol; in particular, and in contrast to some previous work, our protocol does *not* require either party to pre-share a public key. The protocol is also remarkably

The Work of J. Katz was done in part while at Columbia University and Telcordia Technologies. The work of R. Ostrovsky was done in part while at Telcordia Technologies.

J. Katz was supported in part by NSF grant #0627306. R. Ostrovsky was supported in part by a gift from Teradata, an Intel equipment grant, NSF grant #0430254, an OKAWA research award, the B. John Garrick Foundation, and a Xerox Innovation Group award.

Author’s address: J. Katz, Department of Computer Science, University of Maryland, College Park, MD 20742, e-mail: jkatz@umd.edu; R. Ostrovsky, Department of Computer Science, UCLA, Los Angeles, CA 90095-1596, e-mail: rafail@cs.ucla.edu; M. Yung, Department of Computer Science, Columbia University and Google, Inc., 76 Ninth Avenue, 4th Floor, New York, NY 10011; e-mail: moti@cs.columbia.edu.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.

© 2009 ACM 0004-5411/2009/11-ART3 \$10.00

DOI 10.1145/1613676.1613679 <http://doi.acm.org/10.1145/1613676.1613679>

efficient, requiring computation only (roughly) 4 times greater than “classical” Diffie-Hellman key exchange that provides no authentication at all. Ours is the first protocol for password-only authentication that is both *practical* and *provably-secure using standard cryptographic assumptions*.

Categories and Subject Descriptors: C.2.0 [Computer-Communication Networks]: General—Security and protection; F.m [Theory]: Miscellaneous; K.6.5 [Management of Computing and Information Systems]: Security and Protection—Authentication

General Terms: Security, Theory

Additional Key Words and Phrases: Authentication, cryptography, passwords

ACM Reference Format:

Katz, J., Ostrovsky, R., and Yung, M. 2009. Efficient and secure authenticated key exchange using weak passwords. J. ACM 57, 1, Article 3 (November 2009), 39 pages.
DOI = 10.1145/1613676.1613679 <http://doi.acm.org/10.1145/1613676.1613679>

1. Introduction

Protocols for mutual authentication of two parties and generation of a cryptographically-strong shared key between them (*authenticated key exchange*) are fundamental for achieving secure communication over public, insecure networks. Protocols for mutual authentication are necessary because one needs to know “with whom one is communicating”, while key-exchange protocols are required because private-key encryption schemes and message authentication codes rely on shared cryptographic keys that must be refreshed periodically. Higher-level protocols are frequently developed and analyzed assuming the existence of “secure channels” between all parties, yet this assumption cannot be realized without a secure mechanism for implementing such channels using previously-shared information.

The importance of secure key exchange was recognized as early as the seminal work of Diffie and Hellman [1976], which shows how two parties can share a cryptographically-strong key that remains hidden from any *passive* eavesdropper. The Diffie-Hellman protocol, however, does not provide any form of authentication (i.e., a guarantee that the intended partners are sharing the key with *each other*), and in particular it does not protect against an *active* adversary who may inject messages, impersonate one (or both) of the parties, or otherwise control the communication in the network. Achieving any form of authentication inherently requires some information to be shared between the communicating parties in advance of the authentication stage. Historically, authentication protocols were designed under the assumption that the shared information takes the form of high-entropy cryptographic keys: either a *secret key* that can be used for symmetric-key encryption or message authentication, or *public keys* (exchanged by the parties, while the corresponding private keys are kept secret) which can be used for public-key encryption or digital signatures. Extensive attention has been given to the problem of designing two-party authentication and authenticated key-exchange protocols under such assumptions, and a number of provably-secure protocols relying on shared cryptographic keys are known [Bird et al. 1993; Diffie et al. 1992; Bellare and Rogaway 1994; Bellare et al. 1998; Shoup 1999; Canetti and Krawczyk 2001, 2002].

Protocols that assume cryptographically strong pre-shared information simply do not apply—or are trivially seen to be insecure—when the parties share a “short”, low-entropy password instead. As one trivial example to indicate the difficulties that arise, consider the simple (unidirectional) authentication protocol in which

one party sends a random nonce r and the other party replies with $y = H(pw, r)$, where pw represents the shared password and H is a cryptographic hash function. While this protocol can be proven secure (if H is modeled as a random oracle) when pw has high entropy, it is completely insecure when the entropy of pw is small. Indeed, in the latter case a passive adversary who obtains a single transcript (r, y) can recover the password by mounting an *off-line dictionary attack*, trying all values of pw' until one satisfying $y = H(pw', r)$ is found. Other naïve attempts at “bootstrapping” a cryptographic key from a weak password can also be shown to be insecure; as an example, using pw as a “seed” to generate a public key VK for a secure signature scheme also enables an off-line dictionary attack if VK is sent in the clear, or even if a signature is sent in the clear (as an adversary can enumerate all potential public keys and attempt to verify the signature with respect to each of them). Note that “scrambling” pw by, say, hashing it will also be of no help, as the entropy of the hashed output cannot be greater than the entropy of its input.

The above represents a serious problem in practice, as it is well known that most users choose passwords poorly [Morris and Thompson 1979; Klein 1990; Spafford 1992; Wu 1999]. Unfortunately, it has proven difficult to design password-based protocols offering rigorous security guarantees, and there is a long history of protocols for this setting being proposed and subsequently broken (cf. the attacks shown in Bellare and Merritt [1992], Patil [1997], and MacKenzie et al. [2000]). Theoretical progress toward developing provably-secure solutions for the password-based setting has been slow, with the first formal models and proofs of security appearing only recently (see below). The problem is difficult in part because, as we have mentioned above, it requires “bootstrapping” from a weak shared secret to a strong one; furthermore, due to the strong adversarial model considered (cf. Sections 2.1 and 2.2), it is not even *a priori* clear that a solution is possible.

Initial consideration of *password-based authentication* assumed a “hybrid” model in which the client stores the server’s public key in addition to sharing a password with the server. In this setting, Lomas et al. [1989] and Gong et al. [1993] were the first to present authentication protocols with heuristic resistance to off-line dictionary attacks. Formal definitions and rigorous proofs of security in this setting were first given by Halevi and Krawczyk [1999]; see also Boyarsky [1999].

The above-described “hybrid” setting suffers from the disadvantage that the client must store the server’s public key (and if the client will need to authenticate to multiple servers, the client must store multiple public keys). In some sense, this obviates the reason for considering password-based protocols in the first place: namely, that human users cannot remember or securely store long, high-entropy keys. This drawback has motivated research on *password-only* protocols in which the client needs to remember *only* a (short) password. Bellare and Merritt [1992] were the first to consider this, more challenging, setting: they show a number of attacks that can arise, and introduce a set of protocols for so-called “encrypted key exchange” (EKE) that have formed the basis for much future work in this area [Bellare and Merritt 1993; Gong 1995; Steiner et al. 1995; Jablon 1997; Lucks 1997; Wu 1998]. Each of the works just mentioned, however, provides only informal security arguments for the protocols they propose; in fact, attacks against many of these protocols have been shown [Patil 1997; MacKenzie et al. 2000], emphasizing the need for rigorous proofs of security in a formal, well-defined model.

Formal models of security for the password-only setting were given independently by Bellare et al. [2000] (building on Bellare and Rogaway [1994, 1995] and Lucks [1997]) and Boyko, et al. [2000] and MacKenzie et al. [2000] (building on Bellare et al. [1998] and Shoup [1999]). Bellare et al. [2000] also present a proof of security for the two-flow protocol at the “core” of Diffie-Hellman-based EKE in the *ideal cipher model*,¹ while Boyko et al. [2000] introduce new RSA- and Diffie-Hellman-based protocols, and prove their security in the *random oracle model*.² Subsequently, Goldreich and Lindell [2006] introduced a different model of security for this setting and showed the first far, only — protocol for password-only key exchange which is provably secure under standard cryptographic assumptions (and no ideal ciphers/random oracles); their protocol does not require any additional setup beyond the passwords shared by the parties. This work is remarkable in that it shows that password-based authentication is possible under very weak assumptions. On the other hand, their protocol does not lead to a practical solution as it requires techniques from generic two-party secure computation (making their protocol computationally inefficient) and concurrent zero-knowledge (making the round-complexity of their protocol prohibitive). Thus, their protocol should best be viewed as a feasibility result that does not settle the question of whether an *efficient* and *practical* solution is possible. Note that efficiency is particularly important in this setting, which is motivated by inherently practical considerations: that is, human users’ inability to remember long keys. We also remark that the Goldreich-Lindell protocol, unlike the protocol presented here, does not tolerate concurrent executions of the protocol by the same party.

1.1. OUR CONTRIBUTIONS. As the above discussion indicates, previous protocols for password-only key exchange can be classified as either (1) having no rigorous proof of security; (2) having a rigorous proof of security based on standard cryptographic assumptions, but impractical; or (3) having a rigorous proof of security in an idealized model (such as the ideal cipher or random oracle models). We have already discussed why proofs of security are crucial in this setting, and why developing *practical* solutions to the problem of password-only key exchange is of great importance. As for the third class of solutions, it is widely recognized (see, e.g., Bellare and Rogaway [1993], Canetti et al. [2004], and Katz and Lindell [2007]) that proofs of security in the various idealized models discussed earlier are useful as a “sanity check” insofar as they lend a measure of confidence to protocols whose security would otherwise only be heuristic, however, they do *not* provide an “iron-clad” guarantee of security in the real world. As an example [Canetti et al. 2004], a signature scheme is known which can be proven secure in these idealized models but for which *any instantiation of the scheme in the real world* (i.e., where the random oracle is instantiated by a cryptographic hash function) *is insecure*. Thus, proofs of security in idealized models are at best unsatisfying.

We present here the first protocol for password-only key exchange which is both *practical* and *provably-secure under standard cryptographic assumptions*. Security

¹The ideal cipher model assumes a public “oracle” that implements a random keyed, invertible permutation. See further discussion in Section 1.1.

²The random oracle model [Bellare and Rogaway 1993] assumes a public “oracle” that implements a random function; in practice, this oracle is instantiated with a cryptographic hash function. See further discussion in Section 1.1.

is proven in the model of Bellare et al. [2000] based on the decisional Diffie-Hellman assumption [Diffie and Hellman 1976], a well-studied cryptographic assumption used in constructing and analyzing previous protocols for authenticated key-exchange (in the password-based setting and otherwise). Our protocol is remarkably efficient even when compared to the original key-exchange protocol of Diffie and Hellman—which, as we have noted, provides no authentication at all. Our protocol uses only three rounds of communication, and requires computation only (roughly) four times that of the Diffie-Hellman scheme.

Our protocol assumes public parameters available to all parties in the network. These parameters are assumed to be generated in a trusted manner by, say, the same entity who writes and deploys the software implementing the protocol. (This entity is anyway being trusted to implement the protocol correctly, and not to embed any malware or covert channels into the implementation.) The assumption of public parameters is significantly weaker—in both a theoretical and practical sense—than the “hybrid” model discussed earlier in which clients are required to store a public key for each server with whom they wish to communicate or, alternatively, to store the public key of a certificate authority (CA) who certifies servers’ public keys. Advantages of using public parameters as opposed to public keys include:

- The parameters required by our protocol can be generated in such a way that there is no “trapdoor” information associated with them. This, in turn, implies that there is no “master” secret key whose compromise would render the entire system insecure.
- Avoiding the use of public keys avoids the need to handle revocation, and also eliminates the need to transmit certificates and verify certificate chains.
- Once the public parameters are established, *any* client and server who share a password can use the protocol without having to obtain a certificate from a CA.

As an additional contribution, we propose a formal definition of *forward secrecy* for password-only key-exchange protocols that simplifies and unifies previous definitional work in this area. Roughly speaking, forward secrecy guarantees that a protocol remains secure even against an adversary who may obtain some clients’ passwords and who may also change the passwords stored at various servers. It also guarantees protection against malicious clients (who may choose “bad” passwords in an attempt to learn other users’ passwords). As an indication of what can go wrong in this stronger adversarial model, we show that a seemingly innocuous change to our protocol allows for an explicit attack by such an adversary. We then prove, however, that our protocol as specified does indeed achieve forward secrecy.

1.2. SUBSEQUENT PROGRESS. Due in large part to the practical significance and importance of password-based authentication, active research continues in this area. We only highlight those results most related to what is shown here. Perhaps most interesting in this regard is the work of Gennaro and Lindell [2006], who make explicit the intuition underlying the core ideas of our construction and also present and prove secure an abstraction (as well as a generalization) of the framework we use; as corollaries of their work they obtain efficient protocols based on alternative number-theoretic assumptions. Nguyen and Vadhan [2008] show a simpler version of the Goldreich-Lindell protocol which achieves a weaker level of security than that considered in either [Goldreich and Lindell 2006] or here. Though their protocol is somewhat more efficient than that of Goldreich and Lindell [2006],

it relies on generic two-party secure computation and is therefore still not nearly efficient enough to be used in practice. Jiang and Gong [2004] show an efficient protocol for password-based key exchange based on the decisional Diffie-Hellman assumption; the main advantage of their protocol in comparison to the one presented here is that their protocol achieves explicit mutual authentication in three rounds (whereas the protocol shown here would require an additional, fourth round). A stronger definition of security for password-based key exchange has been proposed [Canetti et al. 2005]; a protocol satisfying that stronger definition, building on the work here and in Gennaro and Lindell [2006], is shown there as well. The above (in addition to Goldreich and Lindell [2006] and the present work) are the only schemes currently known with proofs of security under standard cryptographic assumptions.

More efficient variants of our protocol are suggested in Canetti et al. [2005], Katz et al. [2005], and Gennaro [2008]. Threshold variants of our protocol (in which the password is shared across multiple servers to defend against server compromise) have also been suggested [Di Raimondo and Gennaro 2003; Katz et al. 2005].

2. Definitions and Preliminaries

We begin with an informal overview of the model of security used here, intended primarily for those unfamiliar with this area. Section 2.2 contains a formal specification of the security model, following [Bellare et al. 2000]. In Section 2.3, we propose an extension to the basic security model which enables a rigorous definition of *forward secrecy* [Diffie et al. 1992] in the password-only setting. Section 2.4 reviews a number of cryptographic components used by our protocol.

2.1. INFORMAL OVERVIEW OF THE ADVERSARIAL MODEL. In our setting, two parties within a larger network who share a weak (low-entropy) password wish to authenticate each other and generate a cryptographically-strong session key to secure their future communication. They wish to do so in the presence of a powerful adversary who controls *all* communication in the network; this adversary may view, tamper with, deliver out-of-order, or refuse to deliver messages sent by the honest parties. The adversary may also initiate concurrent (arbitrarily-interleaved) executions of the protocol between the honest parties; during these executions, the adversary may attempt to impersonate one (or both) of the parties or may simply eavesdrop on honest executions of the protocol. The adversary may also expose multiple previous session keys in a way we describe below. The adversary succeeds (informally) if it can distinguish any session key generated by an honest party from a randomly chosen session key.³

The description above corresponds to the “basic” adversarial model formalized in Section 2.2. We may additionally consider an adversary who can corrupt parties and thereby either (1) obtain a client’s password, or (2) modify the client’s password stored on a server. We consider this type of adversary when defining “forward secrecy” in Section 2.3.

³The adversary does not succeed by simply “disrupting” the protocol and preventing the parties from agreeing on a shared key; in our model such an attack is impossible to prevent, as the adversary can simply block *all* communication between the honest parties. Note also that we are primarily concerned with the setting of *implicit* authentication; see Section 2.2.1 for a discussion of *explicit* authentication.

A notion of security in the password-based setting must be defined carefully. Since, by their nature, passwords are chosen from a small space of possibilities, an adversary can always try each possible password one-by-one in an *on-line impersonation* attack; once the correct password is found, the adversary can clearly succeed in impersonating either party. With this in mind, we say a password-only protocol is secure if an exhaustive, on-line guessing attack of this sort is the *best* an adversary can do (cf. Definition 1). In particular, a secure protocol is resistant to *off-line* attacks in which an adversary passively monitors executions of the protocol and attempts to determine the correct password by examining the transcripts of these executions. A protocol satisfying our definition of security is also resilient to other types of attacks: for example, the definition rules out “man-in-the-middle” attacks in which an adversary shares a (separate) key with each party although the parties believe they have shared a (single) key with each other.

From a practical point of view, on-line attacks are the hardest for an adversary to mount, and they are also the easiest to detect. Furthermore, on-line attacks may be limited by, for example, shutting down a user’s account after a certain number of failed authentication attempts. It is therefore reasonable to assume that the number of on-line attacks an adversary is able to execute is severely limited, while other attacks (e.g., eavesdropping, off-line password guessing) are not. In any case, as noted above, *any* password-based protocol can ultimately be broken by an on-line attack and so the present definition is the best one can hope for in the password-based setting.

2.2. DEFINITION OF SECURITY FOR PASSWORD-BASED AKE. We essentially follow the definition of Bellare et al. [2000], which is based on prior work by Bellare and Rogaway [1994, 1995] in the non password setting.

Participants, Passwords, and Initialization. We assume a fixed set⁴ of protocol participants (also called principals or users) each of which is either a client $C \in \text{Client}$ or a server $S \in \text{Server}$, where **Client** and **Server** are disjoint. Each $C \in \text{Client}$ is assumed to have a password pw_C , while each $S \in \text{Server}$ is assumed to have a vector $PW_S = \langle pw_{S,C} \rangle_{C \in \text{Client}}$ which contains the passwords of each of the clients. (The assumption that every client shares a password with every server is made merely for convenience. One could just as well assume that arbitrary pairs of users share passwords.)

Prior to any execution of the protocol we assume that an initialization phase occurs during which public parameters (if any) are established and passwords pw_C are chosen for each client C . Recall that pw_C is assumed to be a low-entropy secret; therefore, we assume that pw_C (for each client C) is chosen independently and uniformly at random⁵ from a “dictionary” D of size N , where N is a fixed constant which is independent of the security parameter. The correct passwords

⁴This is for simplicity only, and one could augment the model to allow the adversary to dynamically add new users (and choose their names) during the course of the experiment. By examining the proof, it is easy to see that our protocol remains secure in such a scenario.

⁵The assumption of independent, uniform passwords from a constant-size dictionary is for simplicity only, and our proof of security extends to other cases such as dictionaries whose size depends on the security parameter, non-uniform distributions on passwords, or different distributions for different clients (of course, Definition 1 must be suitably modified in each case). See Section 2.2.1. An even more general model for password selection is considered by Canetti et al. [2005].

are then stored at each server so that $pw_{S,C} = pw_C$ for all $C \in \text{Client}$ and $S \in \text{Server}$.

In general, it is possible for additional information to be generated during this initialization phase. For example, in the “hybrid” model [Halevi and Krawczyk 1999; Boyarsky 1999] public/secret key pairs are generated for each server and the secret key is given as input to the appropriate server, while the public key is provided to all participants (and is also given to the adversary). For the protocol presented here, we require only the weaker requirement of a single set of public parameters which are provided to each party and also to the adversary.

Execution of the Protocol. In the real world, a protocol determines how principals behave in response to input from their environment. In the formal model, these inputs are provided by the adversary. Each principal is assumed to be able to execute the protocol multiple times (possibly concurrently) with different partners; this is modeled by allowing each principal to have an unlimited number of *instances* [Bellare and Rogaway 1995; Bellare et al. 2000] with which to execute the protocol. We denote instance i of user U as Π_U^i . A given instance may be used only once. The adversary is given oracle access to these different instances; furthermore, each instance maintains (local) state which is updated during the course of the experiment. In particular, each instance Π_U^i has associated with it the following variables, initialized as NULL or FALSE (as appropriate) during the initialization phase:

- sid_U^i , pid_U^i , and sk_U^i are variables containing the *session id*, *partner id*, and *session key* for an instance, respectively. Computation of the session key is, of course, the ultimate goal of the protocol. The session id is simply a way to keep track of the different executions of a particular user U ; without loss of generality, we simply let this be the (ordered) concatenation of all messages sent and received by instance Π_U^i . The partner id denotes the identity of the user with whom Π_U^i believes it is interacting. (Note that pid_U^i can never equal U .)
- acc_U^i and term_U^i are Boolean variables denoting whether a given instance has accepted or terminated, respectively. Termination means that an instance is done sending and receiving messages; acceptance indicates successful termination, implying, in particular, that the instance believes it has established a session key with its intended partner. Thus, when an instance Π_U^i accepts sid_U^i , pid_U^i , and sk_U^i are no longer null.
- state_U^i records any state necessary for execution of the protocol by Π_U^i .
- used_U^i is a Boolean variable denoting whether an instance has begun executing the protocol; this formalism ensures that each instance is used only once.

The adversary is assumed to have complete control over all communication in the network. The adversary’s interaction with the principals (more specifically, with the various instances) is modeled via access to *oracles* which we describe now. The state of an instance may be updated during an oracle call, and the oracle’s output may depend upon the state of the relevant instance. The oracle types are:

- Send**(U, i, M): This sends message M to instance Π_U^i . Assuming $\text{term}_U^i = \text{FALSE}$, this instance follows the protocol specification, updating state as appropriate. The output of Π_U^i (i.e., the message sent by the instance) is given to the adversary, who also receives the updated values of sid_U^i , pid_U^i , acc_U^i , and term_U^i .

- Execute**(C, i, S, j): If Π_C^i and Π_S^j have not yet been used (where $C \in \text{Client}$ and $S \in \text{Server}$), this oracle executes the protocol between these instances and outputs the transcript of this execution. In addition to the transcript, the adversary receives the values of **sid**, **pid**, **acc**, and **term**, for both instances, at each step of protocol execution. This oracle call represents passive eavesdropping.
- Reveal**(U, i): This outputs the current value of session key sk_U^i . This oracle call models possible leakage of session keys due to, for example, improper erasure of session keys after use, compromise of a host computer, or cryptanalysis.
- Test**(U, i): This does not model any real-world capability of the adversary, but is used to define security. A random bit b is generated; if $b = 1$ the adversary is given sk_U^i , and if $b = 0$ the adversary is given a random session key. The adversary is allowed only a single **Test** query, at any time during its execution.

Partnering. Let $C \in \text{Client}$ and $S \in \text{Server}$. We say that instances Π_C^i and Π_S^j are *partnered* if: (1) $\text{sid}_C^i = \text{sid}_S^j \neq \text{NULL}$; and (2) $\text{pid}_C^i = S$ and $\text{pid}_S^j = C$. The notion of partnering will be fundamental in defining both correctness and security.

Correctness. To be viable, a key-exchange protocol must satisfy the following notion of correctness: if Π_C^i and Π_S^j are partnered then $\text{acc}_C^i = \text{acc}_S^j = \text{TRUE}$ and $\text{sk}_C^i = \text{sk}_S^j$ (i.e., they both accept and conclude with the same session key).

Advantage of the Adversary. Informally, the adversary succeeds if it can guess the bit b used by the **Test** oracle. Before formally defining the adversary's success, we must first define a notion of *freshness*. An instance Π_U^i is *fresh* unless one of the following is true at the conclusion of the experiment: (1) at some point, the adversary queried **Reveal**(U, i); or (2) at some point, the adversary queried **Reveal**(U', j), where $\Pi_{U'}^j$ and Π_U^i are partnered. We will allow the adversary to succeed only if its **Test** query is made to a fresh instance. Note that this is necessary for any reasonable definition of security; otherwise, the adversary could always succeed by, for example, submitting a **Test** query for an instance for which it had already submitted a **Reveal** query.

We say an adversary $\mathcal{A}$ *succeeds* if it makes a single query **Test**(U, i) to a fresh instance Π_U^i , with $\text{acc}_U^i = \text{TRUE}$ at the time of this query, and outputs a bit b' with $b' = b$ (recall that b is the bit chosen by the **Test** oracle). We denote this event by **Succ**. The advantage of adversary $\mathcal{A}$ in attacking protocol P is then given by:

$$\text{Adv}_{\mathcal{A}, P}(k) \stackrel{\text{def}}{=} 2 \cdot \Pr[\text{Succ}] - 1,$$

where the probability is taken over the random coins used by the adversary as well as those used during the experiment (including the initialization phase).

It remains to define what we mean by a secure protocol. Note that a probabilistic polynomial-time (PPT) adversary can always succeed by trying all passwords one-by-one in an on-line impersonation attack; this is possible since the size of the password dictionary is constant. Informally, a protocol is secure if this is the best an adversary can do. Formally, an instance Π_U^i represents an *on-line attack* if both the following are true at the time of the **Test** query: (1) at some point, the adversary queried **Send**($U, i, *$); and (2) at some point, the adversary queried **Reveal**(U, i) or **Test**(U, i). In particular, instances with which the adversary interacts via **Execute** queries are not counted as on-line attacks. The number of on-line attacks represents

a bound on the number of passwords the adversary could have tested in an on-line fashion. This motivates the following definition:

Definition 2.1. Protocol P is a **secure protocol for password-only authenticated key-exchange** if, for all dictionary sizes N and for all PPT adversaries $\mathcal{A}$ making at most $Q(k)$ on-line attacks, there exists a negligible function $\varepsilon(\cdot)$ such that:

$$\text{Adv}_{\mathcal{A},P}(k) \leq Q(k)/N + \varepsilon(k).$$

The above definition ensures that the adversary can (essentially) do no better than guess a single password during each on-line attack. Calls to the **Execute** oracle, which are not included in $Q(k)$, are of no help to the adversary in breaking the security of the protocol; this means that passive eavesdropping and off-line dictionary attacks are of no use.

We remark that some definitions of security (e.g., Bellare et al. [2000], Goldreich and Lindell [2006], and Nguyen and Vadhan [2008]) allow the adversary to guess more than one password per on-line attempt. We believe the strengthening given by the above definition (in which the adversary can guess only a *single* password per on-line attempt) is important. The space of possible passwords is assumed small to begin with, so any degradation in security should be avoided if possible. Interestingly, for at least one protocol [Wu 1998] an *explicit* attack is known that allows an adversary to guess two passwords per on-line attack.

2.2.1. Extensions of the Definition. We briefly discuss two extensions of the previous definition.

More General Password Distributions. As noted in footnote 5, it is possible to consider more general classes of password distributions. For example, instead of assuming that passwords are chosen uniformly from a fixed dictionary D , we may instead take D to be a probabilistic polynomial-time algorithm that, on input 1^k , outputs a password pw . This allows both for non-uniform distributions, as well as a dependence on the security parameter. For a set S , let $\text{weight}_{D(1^k)}(S) \stackrel{\text{def}}{=} \Pr[D(1^k) \in S]$ and define

$$\text{best-guess}_{D(1^k)}(Q) \stackrel{\text{def}}{=} \max_{S: |S|=Q} \{\text{weight}_{D(1^k)}(S)\}.$$

Then, Definition 2.1 would be modified, in the obvious way, to require that

$$\text{Adv}_{\mathcal{A},P}(k) \leq \text{best-guess}_{D(1^k)}(Q(k)) + \varepsilon(k).$$

It can be verified easily that the proof of security given for our protocol extends to the case where passwords are chosen as above. The requirement that passwords are generated by a *polynomial-time* algorithm D seems necessary for our proof, but it is not necessary for $\text{best-guess}_{D(1^k)}(\cdot)$ to be efficiently computable.

Explicit Mutual Authentication. The definitions we have given are intended only for protocols achieving *implicit*, rather than *explicit*, authentication. (Indeed, the protocol we present here achieves only implicit authentication.) If an honest party S interacts with some party U (who is either the intended partner C or an adversarial impersonator) then, very roughly speaking, S *implicitly authenticates* C if the session key sk that is generated by S is known to U if and only if $U = C$. In

contrast, S *explicitly authenticates* C if the output (sk, acc) of S is such that sk is known to U and $\text{acc} = 1$ if and only if $U = C$.

This means that in the case of protocols for explicit authentication an adversary successfully attacks the protocol if it can either determine the session key held by some instance, or if it can cause an instance to output $\text{acc} = \text{TRUE}$ even though that instance is not partnered with any other instance. A subtlety here is that partnering must be redefined so that instances are considered partnered if their session id's match except possibly for the final message; otherwise it would be trivial for an adversary to succeed by simply forwarding all messages of the protocol *except the last* between two honest parties. (See Bellare and Rogaway [1994] and Bellare et al. [2000] for further discussion.) We also now say that an instance Π_U^i represents an on-line attack if the adversary ever queried $\text{Send}(U, i, *)$ (i.e., without requiring the adversary to also make a **Reveal** or **Test** query for this instance).

Using pseudorandom functions in a standard way, it is easy to add explicit authentication to any protocol achieving implicit authentication [Bellare et al. 2000].

2.3. FORWARD SECRECY. The security definition of the previous section does not offer protection against an adversary who may compromise, say, a server and thereby either (1) obtain the password of a particular client, or (2) modify the value of a particular client's password stored by the server. Building on the framework of Bellare et al. [2000], we provide a definition of *forward secrecy* which addresses these concerns. (Note that forward secrecy [Diffie et al. 1992] classically refers to ensuring security in case long-term secret information—e.g., passwords—are exposed. Following Bellare et al. [2000], however, we additionally allow the adversary to *modify* long-term secret information.) A number of definitions of forward secrecy in the password-only setting have been proposed (e.g., four different notions of forward secrecy are mentioned in Bellare et al. [2000] alone); we believe our definition here simplifies and unifies previous definitional work.

To define forward secrecy, we must modify the basic model of the previous section in two orthogonal ways. First, we augment the adversarial model by introducing a new **Corrupt** oracle to which the adversary will be given access (actually, we will introduce two types of **Corrupt** oracles); this will allow us to model the adversarial actions listed above. Second, we modify the definition of security to obtain a meaningful definition (e.g., once an adversary obtains a client's password it can trivially impersonate that client regardless of what protocol is used; this must be reflected somehow in the security definition). Actually, we will only modify our definition of freshness; the rest of the definition will remain the same. We now discuss each of these changes in turn.

The Corruption Model. We introduce two new oracles to model the two types of attacks listed above. Oracle **Corrupt**₁ models the adversary's ability to learn clients' passwords: formally, **Corrupt**₁(C) returns pw_C for $C \in \text{Client}$.⁶ Oracle **Corrupt**₂ models the adversary's ability to modify passwords stored on a server: formally, **Corrupt**₂(S, C, pw) (for $S \in \text{Server}$ and $C \in \text{Client}$) sets $\text{pw}_{S,C} := \text{pw}$. We emphasize that the adversary can install different passwords on different servers

⁶Note that it does not matter here whether the adversary obtains the password by corrupting a server and learning a client's password stored thereon, or whether the adversary obtains the password directly from the client using, for example, social engineering.

for the same client. In the definition of Bellare et al. [2000], an adversary who installs a password $pw_{S,C}$ also learns the “actual” password pw_C ; we make no such assumption here (instead, the adversary must make an explicit **Corrupt₁** query to obtain this password). In the case of a poorly administered server, it may be easy to modify clients’ passwords without learning their “actual” passwords. The oracle query **Corrupt₂**(S, C', pw) also models the case of a corrupt client C' who selects his own password pw in an arbitrary manner, and then stores it on server S .

“*Freshness*”. Introduction of the **Corrupt** oracles necessitates new definitions of partnering, freshness, and on-line attacks. Let $C \in \text{Client}$ and $S \in \text{Server}$. We now say that instances Π_C^i and Π_S^j are *partnered* if: (1) $\text{sid}_C^i = \text{sid}_S^j \neq \text{NULL}$; (2) $\text{pid}_C^i = S$ and $\text{pid}_S^j = C$; and (3) $pw_C = pw_{S,C}$ at the time these instances were activated. (This last requirement is now necessary since an adversary can cause $pw_C \neq pw_{S,C}$ using a **Corrupt₂** query.) We say that an instance Π_U^i (with $\text{pid}_U^i = U'$) is *fresh* unless one of the following is true at the conclusion of the experiment: (1) at some point, the adversary queried **Reveal**(U, i) or **Reveal**(U', j) where Π_U^j and Π_U^i are partnered; (2) the adversary queried **Corrupt₁**(U) before a query **Send**($U, i, *$); or (3) the adversary queried **Corrupt₁**(U') or **Corrupt₂**($U, U', *$) before a query **Send**($U, i, *$). The second and third conditions reflect the fact that if an adversary learns the password of some client C , then it can trivially impersonate any server to C , or C to any server; the third condition also reflects the fact that once the adversary sets the value of $pw_{S,C}$ to some known value, it can then impersonate C to S . Finally, we define *on-line attack* as follows:

- An instance Π_C^i with $C \in \text{Client}$ represents an on-line attack if (1) the adversary queried **Send**($C, i, *$) before it queried **Corrupt₁**(C); and (2) the adversary subsequently queried **Reveal**(C, i), **Test**(C, i), or **Corrupt₁**(C).
- An instance Π_S^j with $S \in \text{Server}$ and $\text{pid}_S^j = C$ represents an on-line attack if (1) the adversary queried **Send**($S, i, *$) before it queried either **Corrupt₁**(C) or **Corrupt₂**(S, C); and (2) the adversary subsequently queried **Reveal**(S, i), **Test**(S, i), or **Corrupt₁**(C).

Although the above definition is slightly cumbersome, it exactly matches the intuitive notion of what an on-line attack is: the number of on-line attacks exactly represents the maximum number of “password guesses” the adversary can make, and we do not count as on-line attacks scenarios in which the adversary already “knows” the password being used by an instance (e.g., in case the adversary had already queried **Corrupt₁**(C) before it makes a query **Send**(C, i)).

Having defined partnering, freshness, and on-line attacks, the remainder of the security definition is exactly as in the basic case. That is:

- An adversary $\mathcal{A}$ *succeeds* (denoted by **Succ**) if it makes a single query **Test**(U, i) to a fresh instance Π_U^i , with $\text{acc}_U^i = \text{TRUE}$ at the time of this query, and outputs a single bit b' with $b' = b$.
- The advantage of adversary $\mathcal{A}$ in attacking protocol P is defined as

$$\text{Adv}_{\mathcal{A},P}(k) \stackrel{\text{def}}{=} 2 \cdot \Pr[\text{Succ}] - 1.$$

- Let P be a protocol for password-only authenticated key-exchange that is secure in the basic sense of Definition 2.1. We say P achieves forward secrecy if for

all dictionary sizes N and all PPT adversaries $\mathcal{A}$ making at most $Q(k)$ on-line attacks there exists a negligible function $\varepsilon(\cdot)$ such that:

$$\text{Adv}_{\mathcal{A},P}(k) \leq Q(k)/N + \varepsilon(k).$$

We stress that although the above is syntactically equivalent to the basic definition given earlier, the differences are that $\mathcal{A}$ now has access to the two **Corrupt** oracles, and the notions of “freshness” and “on-line attacks” are modified appropriately.

2.4. CRYPTOGRAPHIC BUILDING BLOCKS. We briefly review the decisional Diffie-Hellman assumption as well as some cryptographic building blocks used in our protocol.

The Decisional Diffie-Hellman (DDH) Assumption. Let $\mathcal{G}$ be an efficient algorithm which, on input 1^k , outputs a description of a cyclic (multiplicative) group $\mathbb{G}$ of prime order q where furthermore $|q| = k$. We associate $\mathbb{G}$ with a description of this group, and assume for simplicity that q is implicit in $\mathbb{G}$. Additionally, we assume that group operations (namely, multiplication, membership testing, and finding a generator) in groups output by $\mathcal{G}(1^k)$ can be done in (expected) polynomial-time in k . Exponentiation and selecting a random group element can then also be done in polynomial time. We defer a concrete example of a candidate algorithm $\mathcal{G}$ to the end of this section.

We define $\tilde{\mathbb{G}} \stackrel{\text{def}}{=} \mathbb{G} \setminus \{1\}$; since the order of $\mathbb{G}$ is prime, $\tilde{\mathbb{G}}$ is exactly the set of generators of $\mathbb{G}$. Given a generator $g \in \tilde{\mathbb{G}}$, a random group element can be selected by choosing $x \in \mathbb{Z}_q$ uniformly at random and computing g^x . If, instead, x is chosen uniformly at random from $\mathbb{Z}_q^*$ we obtain a random generator. Given a group $\mathbb{G}$ as above, we define the set of *Diffie-Hellman tuples* as all tuples of the form $(g, h, g^a, h^a) \in \tilde{\mathbb{G}}^4$ where g and h are generators and $a \in \mathbb{Z}_q^*$. We define the set of *random tuples* as all tuples of the form $(g, h, g^a, h^b) \in \tilde{\mathbb{G}}^4$ where g and h are generators, $a, b \in \mathbb{Z}_q^*$, and furthermore $a \neq b$. Informally, the DDH problem for a group $\mathbb{G}$ is to distinguish Diffie-Hellman tuples from random tuples, and the DDH problem is “hard” in $\mathbb{G}$ if no poly-time algorithm can solve the DDH problem in this group with probability much better than $1/2$ (i.e., guessing at random). To be more formal, we must in fact talk about the hardness of the DDH problem in groups output by $\mathcal{G}$. This leads to the following definition.

Definition 2.2. Given $\mathcal{G}$ as above, and for any algorithm D , define

$$\text{Rand}_{D,\mathcal{G}}(k) \stackrel{\text{def}}{=} \Pr[\mathbb{G} \leftarrow \mathcal{G}(1^k); g, h \leftarrow \tilde{\mathbb{G}}; a \leftarrow \mathbb{Z}_q^*; b \leftarrow \mathbb{Z}_q^* \setminus \{a\} : D(\mathbb{G}, g, h, g^a, h^b) = 1]$$

and

$$\text{DH}_{D,\mathcal{G}}(k) \stackrel{\text{def}}{=} \Pr[\mathbb{G} \leftarrow \mathcal{G}(1^k); g, h \leftarrow \tilde{\mathbb{G}}; a \leftarrow \mathbb{Z}_q^* : D(\mathbb{G}, g, h, g^a, h^a) = 1].$$

We say the DDH problem is hard for $\mathcal{G}$ if $|\text{DH}_{D,\mathcal{G}}(k) - \text{Rand}_{D,\mathcal{G}}(k)|$ is negligible for all PPT algorithms D . The DDH assumption is that there exists a $\mathcal{G}$ for which the DDH problem is hard. If $\mathbb{G}$ is a group output by some $\mathcal{G}$ for which the DDH problem is hard, we will sometimes (informally) say the DDH problem is hard in $\mathbb{G}$.

A standard example of an algorithm $\mathcal{G}$ for which the DDH problem is believed to be hard is the following: on input 1^k choose primes p, q such that $p = 2q + 1$ and $|q| = k$; let $\mathbb{G}$ be the subgroup of quadratic residues in $\mathbb{Z}_p^*$. Note that $\mathbb{G}$ has order q , as desired. Of course, other choices for $\mathcal{G}$ are also possible.

One-Time Signature Schemes. We provide a self-contained definition (following Goldwasser et al. [1988]) of the type of signature scheme required by our protocol.

Definition 2.3. A signature scheme Σ is a triple of PPT algorithms $(\text{Gen}, \text{Sign}, \text{Vrfy})$ such that:

- The key generation algorithm Gen takes as input a security parameter 1^k and returns verification key VK and signing key SK .
- The signing algorithm Sign takes as input a signing key SK and a message m , and returns a signature Sig . We denote this by $\text{Sig} \leftarrow \text{Sign}_{\text{SK}}(m)$.
- The verification algorithm Vrfy is a deterministic algorithm that takes as input a verification key VK , a message m , and a signature Sig and returns a single bit. We denote this by $b = \text{Vrfy}_{\text{VK}}(m, \text{Sig})$.

We require that for all k , all (VK, SK) output by $\text{Gen}(1^k)$, all m , and all Sig output by $\text{Sign}_{\text{SK}}(m)$, we have $\text{Vrfy}_{\text{VK}}(m, \text{Sig}) = 1$.

The following definition describes the level of security which the signature scheme used in our protocol must satisfy. The required level of security is relatively weak; thus, signature schemes meeting this level of security are not only easy to construct, but can also be more efficient than typical signature schemes used in practice (which usually satisfy the stronger security notion introduced by Goldwasser et al. [1988]).

Definition 2.4. $\Sigma = (\text{Gen}, \text{Sign}, \text{Vrfy})$ is a secure one-time signature scheme if the following is negligible for all PPT algorithms $\mathcal{F}$:

$$\Pr[(\text{VK}, \text{SK}) \leftarrow \text{Gen}(1^k); (m, \text{Sig}) \leftarrow \mathcal{F}^{\text{Sign}_{\text{SK}}(\cdot)}(1^k, \text{VK}) : \text{Vrfy}_{\text{VK}}(m, \text{Sig}) = 1],$$

where $\mathcal{F}$ makes only a single query to $\text{Sign}_{\text{SK}}(\cdot)$ and we require that Sig was not previously output by $\text{Sign}_{\text{SK}}(\cdot)$ on input m .

The above corresponds to what is sometimes termed a “strong” signature scheme, in that it is also infeasible for the adversary to output a *different* (valid) signature corresponding to the same message m that it submitted to its signing oracle.

A secure (strong) one-time signature scheme may be constructed based on any one-way function, and hence may be based on the DDH assumption (which implies the discrete-logarithm assumption and hence a one-way function).

Collision-Resistant Hashing. Informally, a function H is collision-resistant if it is infeasible to find any $x \neq x'$ for which $H(x) = H(x')$. Formally, let CRHF be an efficient algorithm which, on input 1^k , outputs a description of an efficient hash function H mapping $\{0, 1\}^*$ to $\{0, 1\}^k$. We say that CRHF is a collision-resistant hash family if the following is negligible for all PPT algorithms A :

$$\Pr[H \leftarrow \text{CRHF}(1^k); (x, x') \leftarrow A(1^k, H) : x \neq x' \wedge H(x) = H(x')].$$

For our application, we will actually require that the H output by CRHF map strings to $\mathbb{Z}_q$ for some given q of length k . However, it is easy to modify any collision-resistant hash family as described above to obtain one satisfying this requirement.

A collision-resistant hash family may be constructed based on the DDH assumption (in fact, the weaker discrete-logarithm assumption suffices) [Damgård 1988].

Cramer-Shoup Encryption Using Labels. The Cramer-Shoup public-key encryption scheme [Cramer and Shoup 2003] was the first known example of a practical encryption scheme with a rigorous proof of security against chosen-ciphertext attacks under standard cryptographic assumptions. Our protocol relies on a modification of the Cramer-Shoup scheme, described here for convenience. For completeness, we also provide a definition of security against chosen-ciphertext attacks incorporating the notion of *labels* (see Shoup [2001]). We rely on this definition in the proof of security for our protocol.

Before continuing, we emphasize that our protocol does *not* use the (modified) Cramer-Shoup scheme for *encryption* per se. Indeed, in our application no party publishes a public key or holds any secret key associated with the scheme, and “decryption” is never performed during execution of our protocol. Our proof of security, however, does rely on the fact that decryption is possible. Gennaro and Lindell [2006] have since shown that this feature is not essential, and that nonmalleable *commitment* [Dolev et al. 2000] suffices.⁷ We first define the semantics of public-key encryption with labels.

Definition 2.5. A public-key encryption scheme (supporting labels) is a tuple of PPT algorithms (KeyGen , $\mathcal{E}$, $\mathcal{D}$) such that:

- The key-generation algorithm KeyGen takes as input a security parameter 1^k and returns a public key pk and a secret key sk .
- The encryption algorithm $\mathcal{E}$ takes as input a public key pk , a label LABEL, and a message m . It returns a ciphertext C . We write this as $C \leftarrow \mathcal{E}_{pk}(\text{LABEL}, m)$.
- The decryption algorithm $\mathcal{D}$ takes as input a secret key sk , a label LABEL, and a ciphertext C . It returns a message m or a distinguished symbol $\perp$. We write this as $m = \mathcal{D}_{sk}(\text{LABEL}, C)$.

We require that for all k , all pk, sk output by $\text{KeyGen}(1^k)$, any LABEL, all m in the (implicit) message space, and any C output by $\mathcal{E}_{pk}(\text{LABEL}, m)$ we have $\mathcal{D}_{sk}(\text{LABEL}, C) = m$.

Our definition of security against (adaptive) chosen-ciphertext attacks is standard [Shoup 2001]. In the following, we define a **left-or-right encryption oracle** $\mathcal{E}_{pk,b}(\cdot, \cdot, \cdot)$ (where $b \in \{0, 1\}$) as $\mathcal{E}_{pk,b}(\text{LABEL}, m_0, m_1) \stackrel{\text{def}}{=} \mathcal{E}_{pk}(\text{LABEL}, m_b)$.

Definition 2.6. A public-key encryption scheme (KeyGen , $\mathcal{E}$, $\mathcal{D}$) is **secure against chosen-ciphertext attacks** if the following is negligible for all PPT algorithms A :

$$\left| 2 \cdot \Pr[(pk, sk) \leftarrow \text{KeyGen}(1^k); b \leftarrow \{0, 1\} : A^{\mathcal{E}_{pk,b}(\cdot, \cdot, \cdot), \mathcal{D}_{sk}(\cdot, \cdot)}(1^k, pk) = b] - 1 \right|,$$

⁷ Security of the Cramer-Shoup scheme against chosen-ciphertext attacks implies that it is a non-malleable commitment scheme in the common reference string model [Di Crescenzo et al. 2001].

where A 's queries are restricted as follows: if A makes a query $\mathcal{E}_{pk,b}(\text{LABEL}, m_0, m_1)$ then m_0, m_1 must be in the (implicit) message space with $|m_0| = |m_1|$; furthermore, if A receives ciphertext C in response to any such query, then A cannot later query $\mathcal{D}_{sk}(\text{LABEL}, C)$.

We now describe our modification of the Cramer-Shoup encryption scheme. On input 1^k , the key-generation algorithm **KeyGen** first runs $\mathcal{G}(1^k)$ to generate a group $\mathbb{G}$ (as described earlier in the context of the DDH assumption); it also selects random generators $g_1, g_2 \leftarrow \mathbb{G}$. Next, it selects three pairs $(z_1, z_2), (x_1, x_2), (y_1, y_2)$ at random from the set $\{(a, b) \in \mathbb{Z}_q \times \mathbb{Z}_q \mid g_1^a g_2^b \neq 1\}$, and sets $h = g_1^{z_1} g_2^{z_2}$, $c = g_1^{x_1} g_2^{x_2}$, and $d = g_1^{y_1} g_2^{y_2}$. Finally, it runs **CRHF** (1^k) to generate a hash function H , where **CRHF** is a collision-resistant hash family as described above. The public key consists of $\mathbb{G}, g_1, g_2, h, c, d, H$; the secret key is $z_1, z_2, x_1, x_2, y_1, y_2$; and the message space is $\mathbb{G}$.

To encrypt a message $m \in \mathbb{G}$ using label LABEL , the sender chooses a random $r \in \mathbb{Z}_q$ and computes $u_1 = g_1^r$, $u_2 = g_2^r$, and $e = h^r m$. It then sets $\alpha = H(\text{LABEL} \parallel u_1 \parallel u_2 \parallel e)$ and outputs the ciphertext $\langle u_1 \parallel u_2 \parallel e \parallel (cd^\alpha)^r \rangle$. To decrypt a ciphertext $\langle u_1 \parallel u_2 \parallel e \parallel v \rangle$ using label LABEL , the receiver first verifies that all components of the ciphertext lie in $\mathbb{G}$. If not, it outputs $\perp$; otherwise, it then computes $\alpha = H(\text{LABEL} \parallel u_1 \parallel u_2 \parallel e)$ and checks whether $u_1^{x_1 + \alpha y_1} u_2^{x_2 + \alpha y_2} \stackrel{?}{=} v$. If not, the decryption algorithm outputs $\perp$; otherwise, it outputs $m = e / u_1^{z_1} u_2^{z_2}$. It is not hard to see that decryption (of honestly-generated ciphertexts) always succeeds.

The following is easily derived from the results of Cramer and Shoup [2003]:

THEOREM 2.7. *Assuming (1) the DDH problem is hard for $\mathcal{G}$ and (2) **CRHF** is a collision-resistant hash family, the above scheme is secure against adaptive chosen-ciphertext attacks.*

3. A Provably-Secure Protocol for Password-Only AKE

3.1. DESCRIPTION OF THE PROTOCOL. A high-level depiction of the protocol is given in Figure 1, and a more detailed description follows. A completely formal specification of the protocol appears in Section 3.2, where we give a proof of security for the protocol in the “basic” adversarial model of Section 2.2. In Section 3.3 we show that the protocol does *not* achieve forward secrecy if it is modified in a seemingly-innocuous way (this is meant to motivate our definition of forward secrecy, as well as to illustrate the importance of rigorous proofs of security). Then, we prove that the protocol as formally defined *does* achieve forward secrecy.

The protocol as described here is different from what appears in previous versions of this work [Katz 2002; Katz et al. 2001, 2002], as we have introduced small modifications in order to simplify the proof of security (the proofs of security given previously are, however, correct). These modifications do not significantly affect the efficiency of the protocol, nor do they necessitate additional assumptions.

Initialization. For security parameter k , the public parameters contain a group $\mathbb{G}$ (written multiplicatively) having prime order q with $|q| = k$; we assume the hardness of the DDH problem in $\mathbb{G}$. Additionally, the parameters include random generators $g_1, g_2, h, c, d \in \mathbb{G}$ and a hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ chosen at random from a collision-resistant hash family.

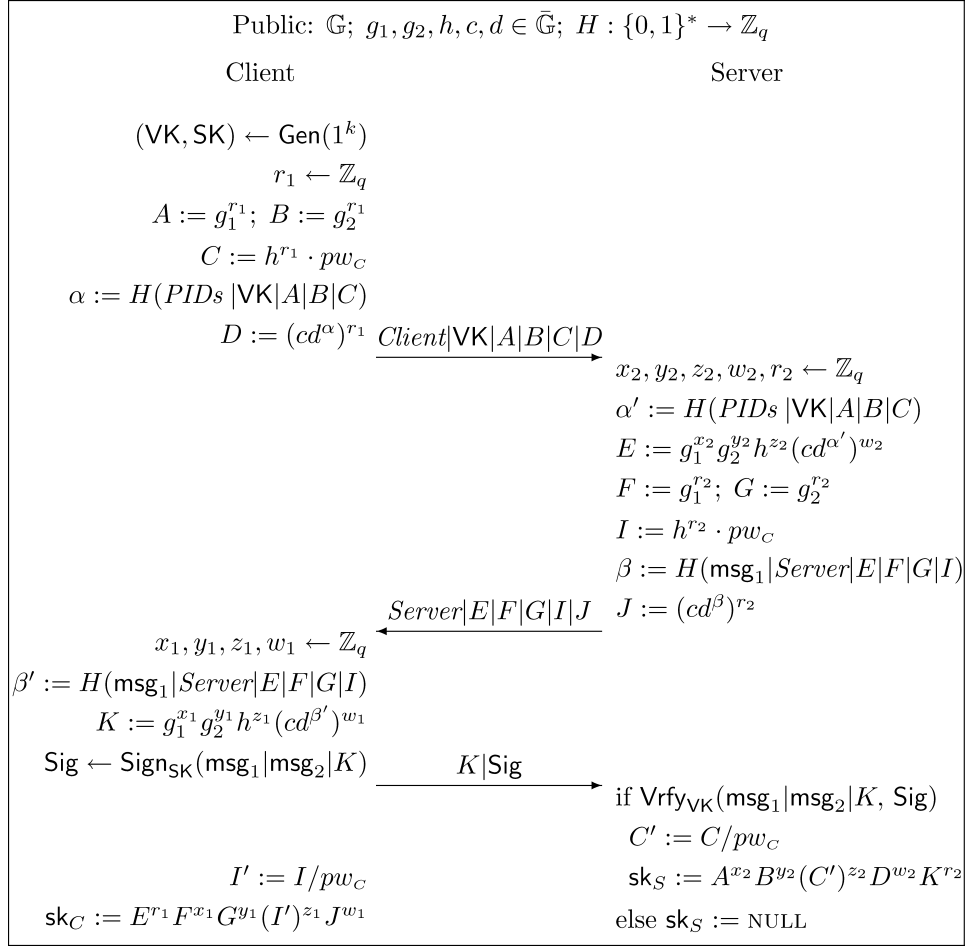


FIG. 1. A protocol for password-only authenticated key exchange. The client name is *Client*, the server name is *Server*, and we let $PIDs := \text{"Client"} \parallel \text{"Server"}$. The first and second messages of the protocol are called msg_1 and msg_2 , respectively.

As part of the initialization, a password pw_C is chosen randomly for each client C , and pw_C is stored by each server (cf. Section 2.2). We assume that all passwords lie in $\mathbb{G}$ or can be mapped in a one-to-one fashion to $\mathbb{G}$; this is typically easy: for example, if passwords are represented as integers less than q then the password pw can be mapped to $g_1^{pw} \in \mathbb{G}$. For ease of exposition, we will simply assume that passwords lie in the set $\{g_1^1, \dots, g_1^N\}$ where N represents the size of the dictionary from which passwords are chosen. (We also implicitly assume that $N < q$, which will certainly be true in practice.)

Protocol Execution. When a client $Client \in \text{Client}$ with password pw_C wants to connect to a server $Server \in \text{Server}$, the client computes a Cramer-Shoup “encryption” of pw_C , as described in Section 2.4, using a particular value for the label. In more detail, let $PIDs$ denote the string “*Client* $\parallel$ *Server*”. The client begins by running a key-generation algorithm for a one-time signature scheme, giving VK and SK . The client chooses random $r_1 \in \mathbb{Z}_q$ and computes $A = g_1^{r_1}$, $B = g_2^{r_1}$,

and $C = h^{r_1} \cdot pw_c$. The client then computes $\alpha = H(PIDs|VK|A|B|C)$ and sets $D = (cd^\alpha)^{r_1}$. The client sends

$$\text{msg}_1 \stackrel{\text{def}}{=} \langle \text{Client}|VK|A|B|C|D \rangle$$

to the server as the first message of the protocol. Note that this corresponds roughly to an “encryption” of pw_c using the label $PIDs|VK$.

Upon receiving the message msg_1 (as above), the server computes a Cramer-Shoup “encryption” of pw_c (where pw_c is the password corresponding to the client named in the incoming message) using a particular value for the label. In more detail, the server first chooses random $x_2, y_2, z_2, w_2 \in \mathbb{Z}_q$, computes $\alpha' = H(PIDs|VK|A|B|C)$, and sets $E = g_1^{x_2} g_2^{y_2} h^{z_2} (cd^{\alpha'})^{w_2}$. Additionally, a random $r_2 \in \mathbb{Z}_q$ is chosen and the server computes $F = g_1^{r_2}$, $G = g_2^{r_2}$, and $I = h^{r_2} \cdot pw_c$. The server then computes $\beta = H(\text{msg}_1|Server|E|F|G|I)$ and sets $J = (cd^\beta)^{r_2}$. The server sends

$$\text{msg}_2 \stackrel{\text{def}}{=} \langle \text{Server}|E|F|G|I|J \rangle$$

to the client as the second message of the protocol. Note that this process corresponds roughly to an “encryption” of pw_c using the label $\text{msg}_1|Server|E$.

Upon receiving the message msg_2 , the client chooses random $x_1, y_1, z_1, w_1 \in \mathbb{Z}_q$, computes $\beta' = H(\text{msg}_1|Server|E|F|G|I)$, and sets $K = g_1^{x_1} g_2^{y_1} h^{z_1} (cd^{\beta'})^{w_1}$. The client then signs $\text{msg}_1|\text{msg}_2|K$ using the secret key SK that it generated in the first step of the protocol. The value K and the resulting signature are sent as the final message of the protocol. At this point, the client accepts and determines the session key by first computing $I' = I/pw_c$ and then setting $sk_c = E^{r_1} F^{x_1} G^{y_1} (I')^{z_1} J^{w_1}$.

Upon receiving $K|\text{Sig}$, the server first checks that Sig is a valid signature of $\text{msg}_1|\text{msg}_2|K$ under VK . If so, the server accepts and determines the session key by computing $C' = C/pw_{s,c}$ and then setting $sk_s = A^{x_2} B^{y_2} (C')^{z_2} D^{w_2} K^{r_2}$. Otherwise, the server terminates without accepting and the session key remains NULL.

Although omitted in the above description, we assume that the client and server always check that incoming messages are well-formed. In particular, when the server receives the first message it verifies that $\text{Client} \in \text{Client}$ and that $A, B, C, D \in \mathbb{G}$ (recall that membership in $\mathbb{G}$ can be efficiently verified). When the client receives the second message, it verifies that the server name included in the message is indeed the name of the server to whom the client desired to connect, and that $E, F, G, I, J \in \mathbb{G}$. Finally, when the server receives the last message it verifies that $K \in \mathbb{G}$ in addition to verifying correctness of the signature. If an ill-formed message is ever received, the receiving party terminates immediately without accepting and the session key remains NULL. For further details, see the formal description of the protocol in Figures 3 and 4, below.

Correctness. In an honest execution of the protocol, the client and the server calculate identical session keys. To see this, note that in an honest execution we have $\alpha = \alpha'$, $\beta = \beta'$, and the same password pw_c is used by both parties. Thus,

$$\begin{aligned} E^{r_1} &= (g_1^{x_2} g_2^{y_2} h^{z_2} (cd^\alpha)^{w_2})^{r_1} \\ &= (g_1^{r_1})^{x_2} (g_2^{r_1})^{y_2} (h^{r_1})^{z_2} ((cd^\alpha)^{r_1})^{w_2} \\ &= A^{x_2} B^{y_2} (C')^{z_2} D^{w_2} \end{aligned}$$

and

$$\begin{aligned} K^{r_2} &= (g_1^{x_1} g_2^{y_1} h^{z_1} (cd^\beta)^{w_1})^{r_2} \\ &= (g_1^{r_2})^{x_1} (g_2^{r_2})^{y_1} (h^{r_2})^{z_1} ((cd^\beta)^{r_2})^{w_1} \\ &= F^{x_1} G^{y_1} (I')^{z_1} J^{w_1}. \end{aligned}$$

Therefore:

$$\text{sk}_c = E^{r_1} (F^{x_1} G^{y_1} (I')^{z_1} J^{w_1}) = (A^{x_2} B^{y_2} (C')^{z_2} D^{w_2}) K^{r_2} = \text{sk}_s$$

and the session keys are equal.

Achieving Explicit Authentication. As described, the protocol does not achieve explicit mutual authentication (i.e., a party does not know whether its intended partner has successfully computed a matching session key). It is easy to add explicit authentication using standard techniques (see, e.g., Bellare et al. [2000]).

Efficiency Considerations. Some remarks about the computational efficiency of the above scheme are in order. First, efficiency of the signature computation can be improved using an on-line/off-line signature scheme [Even et al. 1996], where the off-line computation is done while the client is waiting for the server to respond. Also, the data being signed (namely, $\text{msg}_1 | \text{msg}_2 | K$) could clearly be hashed using H before signature computation. By swapping the roles of the client and server (so that the server sends the first message in Figure 1), the server can use long-term public-/private-keys (for a signature scheme which is existentially unforgeable under adaptive chosen-message attacks [Goldwasser et al. 1988]) and thereby avoid having to run the key-generation algorithm each time the protocol is executed. We stress that in this case *we do not require that the client store the server's long-term public key*; instead, the public key is included in the first message from the server but the protocol will be secure even if an active adversary replaces this key with one of his own choosing.

Algorithms for simultaneous multiple exponentiation [Menezes et al. 1996, Chapter 14] can be used to speed up the computation in the protocol. If this is done, the computation for each user is (roughly) 7–8 exponentiations in $\mathbb{G}$ (ignoring the effects of the signature scheme), as compared to the 2 exponentiations per user in standard Diffie-Hellman key exchange [Diffie and Hellman 1976], which provides no authentication at all.

Further efficiency improvements are discussed in Canetti et al. [2005], Katz et al. [2005], and Gennaro [2008].

3.2. PROOF OF SECURITY. We first provide a formal specification of the protocol by specifying the initialization phase and the oracles to which the adversary has access. During the initialization phase for security parameter k (cf. Figure 2), algorithm `Initialize` first runs an algorithm $\mathcal{G}$ to generate a group $\mathbb{G}$ of prime order q with $|q| = k$. Next, generators $g_1, g_2, h, c, d \in \mathbb{G}$ are selected at random, and a hash function H is chosen from a collision-resistant hash family **CRHF**. Furthermore, the sets **Client** and **Server** are determined using some (arbitrary) algorithm `UserGen`. Passwords for each client are chosen at random as discussed in the previous section; the passwords for each client are then stored at each server.

A formal specification of the `Execute`, `Reveal`, and `Test` oracles appears in Figure 3. The description of the `Execute` oracle matches the high-level protocol description of Figure 1, but additional details (for example, the updating

```

Initialize( $1^k$ ) —
   $\mathbb{G} \leftarrow \mathcal{G}(1^k)$ 
   $g_1, g_2, h, c, d \leftarrow \bar{\mathbb{G}}$ 
   $H \leftarrow \text{CRHF}(1^k)$ 
   $(\text{Client}, \text{Server}) \leftarrow \text{UserGen}(1^k)$ 
  for each  $C \in \text{Client}$ 
     $pw'_C \leftarrow \{1, \dots, N\}$ 
     $pw_C := g_1^{pw'_C}$ 
    for each  $S \in \text{Server}$ 
       $pw_{S,C} := pw_C$ 
  return  $\text{Client}, \text{Server}, \mathbb{G}, g_1, g_2, h, c, d, H$ 

```

FIG. 2. Specification of protocol initialization.

```

Execute( $\text{Client}, i, \text{Server}, j$ ) —
  if ( $\text{Client} \notin \text{Client}$  or  $\text{Server} \notin \text{Server}$  or  $\text{used}_{\text{Client}}^i$  or  $\text{used}_{\text{Server}}^j$ )
    return  $\perp$ 
   $\text{used}_{\text{Client}}^i := \text{TRUE}$ ;  $\text{used}_{\text{Server}}^j := \text{TRUE}$ 
   $\text{PIDs} := \text{Client}|\text{Server}$ ;  $(\text{VK}, \text{SK}) \leftarrow \text{Gen}(1^k)$ 
   $x_1, x_2, y_1, y_2, z_1, z_2, w_1, w_2, r_1, r_2 \leftarrow \mathbb{Z}_q$ 
   $A := g_1^{r_1}$ ;  $B := g_2^{r_1}$ ;  $C := h^{r_1} \cdot pw_{\text{Client}}$ ;  $\alpha := H(\text{PIDs}|\text{VK}|A|B|C)$ 
   $D := (cd^\alpha)^{r_1}$ ;  $\text{msg}_1 := \langle \text{Client}|\text{VK}|A|B|C|D \rangle$ 
   $\text{status}_{\text{Client},1}^i := \langle \text{NULL}, \text{Server}, \text{FALSE}, \text{FALSE} \rangle$ 

   $F := g_1^{r_2}$ ;  $G := g_2^{r_2}$ ;  $I := h^{r_2} \cdot pw_{\text{Server}, \text{Client}}$ ;  $E := g_1^{x_2} g_2^{y_2} h^{z_2} (cd^\alpha)^{w_2}$ 
   $\beta := H(\text{msg}_1|\text{Server}|E|F|G|I)$ ;  $J := (cd^\beta)^{r_2}$ 
   $\text{msg}_2 := \langle \text{Server}|E|F|G|I|J \rangle$ 
   $\text{status}_{\text{Server},1}^j := \langle \text{NULL}, \text{Client}, \text{FALSE}, \text{FALSE} \rangle$ 

   $K := g_1^{x_1} g_2^{y_1} h^{z_1} (cd^\beta)^{w_1}$ ;  $\text{Sig} \leftarrow \text{Sign}_{\text{SK}}(\text{msg}_1|\text{msg}_2|K)$ 
   $\text{msg}_3 := \langle K|\text{Sig} \rangle$ 

   $\text{sid}_{\text{Client}}^i := \text{sid}_{\text{Server}}^j := \langle \text{msg}_1|\text{msg}_2|\text{msg}_3 \rangle$ 
   $\text{pid}_{\text{Client}}^i := \text{Server}$ ;  $\text{pid}_{\text{Server}}^j := \text{Client}$ 
   $\text{acc}_{\text{Client}}^i := \text{term}_{\text{Client}}^i := \text{acc}_{\text{Server}}^j := \text{term}_{\text{Server}}^j := \text{TRUE}$ 
   $\text{sk}_{\text{Client}}^i := E^{r_1} F^{x_1} G^{y_1} (I/pw_{\text{Client}})^{z_1} J^{w_1}$ 
   $\text{sk}_{\text{Server}}^j := A^{x_2} B^{y_2} (C/pw_{\text{Server}, \text{Client}})^{z_2} D^{w_2} K^{r_2}$ 
  return  $\text{msg}_1, \text{msg}_2, \text{msg}_3,$ 
     $\text{status}_{\text{Client},1}^i, \text{status}_{\text{Server},1}^j, \text{status}_{\text{Client}}^i, \text{status}_{\text{Server}}^j$ 

Reveal( $U, i$ ) —
  return  $\text{sk}_U^i$ 

Test( $U, i$ ) —
   $b \leftarrow \{0, 1\}$ ;  $\text{sk}' \leftarrow \mathbb{G}$ 
  if  $b = 0$  return  $\text{sk}'$  else return  $\text{sk}_U^i$ 

```

FIG. 3. Specification of the Execute, Reveal, and Test oracles.

```

Send0(Client, i, Server) —
  if (Client ∉ Client or Server ∉ Server or usedClienti) return ⊥
  usedClienti := TRUE; PIDs := Client|Server; (VK, SK) ← Gen(1k); r ← ℤq
  A := g1r; B := g2r; C := hr · pwClient; α := H(PIDs|VK|A|B|C)
  D := (cdα)r; msg-out := ⟨Client|VK|A|B|C|D⟩
  statusClienti := ⟨NULL, Server, FALSE, FALSE⟩
  stateClienti := ⟨SK, r, msg-out⟩
  return msg-out, statusClienti

Send1(Server, j, ⟨Client|VK|A|B|C|D⟩) —
  if (Server ∉ Server or usedServerj) return ⊥
  usedServerj := TRUE; PIDs := Client|Server
  if A, B, C, D ∉ ℤ or Client ∉ Client
    statusServerj := ⟨NULL, NULL, FALSE, TRUE⟩; return statusServerj
  x, y, z, w, r ← ℤq; α := H(PIDs|VK|A|B|C)
  F := g1r; G := g2r; I := hr · pwServer, Client; E := g1xg2yhz(cdα)w
  β := H(msg-in|Server|E|F|G|I); J := (cdβ)r; msg-out := ⟨Server|E|F|G|I|J⟩
  statusServerj := ⟨NULL, Client, FALSE, FALSE⟩
  stateServerj := ⟨msg-in, x, y, z, w, r, msg-out, pwServer, Client⟩
  return msg-out, statusServerj

Send2(Client, i, ⟨Server|E|F|G|I|J⟩) —
  if (Client ∉ Client or ¬usedClienti or termClienti) return ⊥
  if Server ≠ pidClienti or E, F, G, I, J ∉ ℤ
    statusClienti := ⟨NULL, NULL, FALSE, TRUE⟩; return statusClienti
  ⟨SK, r, first-msg-out⟩ := stateClienti
  x, y, z, w ← ℤq; β := H(first-msg-out|Server|E|F|G|I)
  K := g1xg2yhz(cdβ)w; Sig ← SignSK(first-msg-out|msg-in|K);
  msg-out := ⟨K|Sig⟩
  sidClienti := ⟨first-msg-out|msg-in|msg-out⟩
  accClienti := termClienti := TRUE
  skClienti := ErFxGy(I/pwClient)zJw
  return msg-out, statusClienti

Send3(Server, j, ⟨K|Sig⟩) —
  if (Server ∉ Server or ¬usedServerj or termServerj) return ⊥
  ⟨first-msg-in, x, y, z, w, r, first-msg-out, pw⟩ := stateServerj
  ⟨Client|VK|A|B|C|D⟩ := first-msg-in
  if K ∉ ℤ or VrfyVK(first-msg-in|first-msg-out|K, Sig) ≠ 1
    statusServerj := ⟨NULL, NULL, FALSE, TRUE⟩; return statusServerj
  sidServerj := ⟨first-msg-in|first-msg-out|msg-in⟩
  accServerj := termServerj := TRUE
  skServerj := AxBy(C/pw)zDwKr
  return statusServerj

```

FIG. 4. Specification of the Send oracle (msg-in denotes the third argument to the oracle).

of state information) are included. We let status_U^i denote the vector of values $\langle \text{sid}_U^i, \text{pid}_U^i, \text{acc}_U^i, \text{term}_U^i \rangle$ associated with instance Π_U^i . A formal specification of the Send oracle appears in Figure 4. Although the model technically has only one type of Send oracle (see Section 2.2), the adversary's queries to this oracle can be viewed as queries to four different oracles $\text{Send}_0, \dots, \text{Send}_3$ representing the

four different types of messages which may be sent as part of the protocol (this includes the three message types shown in Figure 1 as well as an “initiate” message). The third argument to each oracle is denoted by `msg-in`. In the proof that follows, we will also sometimes refer to `msg1`, `msg2`, `msg3`, where these refer to messages having the correct format for rounds 1, 2, and 3, respectively. We will also use these to denote messages input to the `Send1`, `Send2`, or `Send3` oracles, respectively, or output by the `Send0`, `Send1`, or `Send2` oracles, respectively.

THEOREM 3.1. *Assuming (1) the DDH problem is hard for $\mathcal{G}$; (2) $(\text{Gen}, \text{Sign}, \text{Vrfy})$ is a secure one-time signature scheme; and (3) CRHF is a collision-resistant hash family, the protocol of Figure 1 is a secure protocol for password-only authenticated key exchange.*

Since one-time signature schemes as well as collision-resistant hash functions may be constructed based on the DDH assumption (cf. Section 2.4), we have the following corollary.

COROLLARY 3.2. *Under the DDH assumption, there exists a secure protocol for password-only authenticated key exchange.*

Before giving the formal details, we describe the high-level structure of the proof of Theorem 3.1. Let P_0 denote the “real-world” experiment where an adversary attacks the actual protocol. We introduce a sequence of transformations to this experiment, and bound the effect of each transformation on the adversary’s advantage. We then bound the adversary’s advantage in the final experiment; this yields a bound on the adversary’s advantage when attacking the original protocol.

In experiment P_2 (obtained by a sequence of modifications to P_0), all session keys computed in response to an `Execute` query are chosen uniformly at random and the password is not used at all; `Send` queries are treated exactly as in P_0 . It is not too difficult to see that the adversary’s advantage cannot change significantly in moving from P_0 to P_2 , since the “core” of an honest execution of the protocol is based on the Diffie-Hellman key-exchange protocol (and `Execute` queries correspond to passive eavesdropping on a protocol execution), and the password is anyway “encrypted” using Cramer-Shoup encryption.

In experiment P_3 , we (informally) consider the adversary to have succeeded as soon as it interacts with a client or server using the “correct” value of the password. (Thus, the adversary may now succeed even without guessing the correct value of b .) This can only increase the advantage of the adversary.

Experiment P_5 (preceded by an intermediate experiment P_4 for technical reasons) is really the crux of the proof. Roughly speaking, we now define the experiment so that whenever the adversary interacts with a server using an *incorrect* value of the password the server chooses a session key uniformly at random. We then prove that this has *no* effect on the adversary’s advantage, in an information-theoretic sense; expressed differently, this means that the session key computed by a server in experiment P_3 —when the adversary interacts with a server using an incorrect value of the password—is uniformly distributed from the adversary’s point of view.

In experiment P_6 , we now have the server compute the first message of the protocol (in response to a `Send` query) independently of the actual password.

Relying on the security of the Cramer-Shoup encryption scheme, we show that this cannot significantly affect the adversary's advantage.

Experiments P_7 and P_8 are analogous to experiments P_5 and P_6 , but focusing on client-instances rather than server-instances.

In experiment P_8 , we notice that the adversary's view is independent of any actual passwords except for the fact that the adversary succeeds (as per the modification introduced in experiment P_3) as soon as it interacts with a party using a "correct" password; this event therefore occurs with probability at most $Q(k)/N$. Furthermore, if this event does not happen then the bit b is information-theoretically hidden from the adversary (since all session keys are chosen uniformly at random), and so the adversary succeeds with probability exactly half. The overall probability that the adversary succeeds in experiment P_8 is thus at most

$$Q(k)/N + \frac{1}{2} \cdot \left(1 - \frac{Q(k)}{N}\right).$$

The analysis of the above sequence of experiments shows that the adversary's success probability in the original experiment is at most negligibly greater than this.

We now give the formal details.

PROOF. Throughout the proof, we will refer to the formal specification of the protocol as it appears in Figures 2–4. It is always the case that $pw_{Server,Client} = pw_{Client}$ for all $Client \in \mathbf{Client}$ and $Server \in \mathbf{Server}$; furthermore, during execution of the $\mathbf{Send}_3$ oracle it is always the case that $pw = pw_{Client}$. (This will not necessarily be true when considering forward secrecy; see the following section.) Given an adversary $\mathcal{A}$, we imagine a simulator that runs the protocol for $\mathcal{A}$. More precisely, the simulator begins by running algorithm $\mathbf{Initialize}(1^k)$ (which includes choosing passwords for clients) and giving the public output of the algorithm to $\mathcal{A}$. When $\mathcal{A}$ queries an oracle, the simulator responds by executing the appropriate algorithm as in Figures 3 and 4; the simulator also records all state information defined during the course of the experiment. In particular, when the adversary queries the $\mathbf{Test}$ oracle, the simulator chooses (and records) the random bit b . When the adversary completes its execution and outputs a bit b' , the simulator can tell whether the adversary succeeds by checking whether (1) a single $\mathbf{Test}$ query was made, for some instance Π_U^i ; (2) $\mathbf{acc}_U^i$ was true at the time of the $\mathbf{Test}$ query; (3) instance Π_U^i is fresh; and (4) $b' = b$. Success of the adversary is denoted by event $\mathbf{Succ}$.

For any experiment P we define $\mathbf{Adv}_{\mathcal{A},P}(k) \stackrel{\text{def}}{=} 2 \cdot \Pr_{\mathcal{A},P}[\mathbf{Succ}] - 1$, where $\Pr_{\mathcal{A},P}[\cdot]$ denotes the probability of an event when the simulator interacts with the adversary $\mathcal{A}$ in accordance with experiment P . We refer to the real execution of the experiment, as described above, as P_0 . We introduce a sequence of transformations to the original experiment and bound the effect of each transformation on the adversary's advantage. We then bound the adversary's advantage in the final experiment; this yields a bound on the adversary's advantage in the original experiment.

We begin with some terminology that will be used throughout the proof. A given $\mathbf{msg}_1$ is called *oracle-generated* if it was output by the simulator in response to some oracle query (whether a $\mathbf{Send}_0$ or $\mathbf{Execute}$ query). This message is said to be *adversarially generated* otherwise. These notions are defined analogously for a given $\mathbf{msg}_2$. A verification key contained in an oracle-generated $\mathbf{msg}_1$ is called an *oracle-generated verification key*.

Experiment P'_0 . In experiment P'_0 , the simulator interacts with the adversary as before except that the adversary does *not* succeed, and the experiment is aborted, if any of the following occur:

- (1) At any point, an oracle-generated verification key is used more than once. In particular, the experiment is aborted if an oracle-generated msg_1 is ever repeated.
- (2) At any point during the experiment, an oracle-generated msg_2 is repeated.
- (3) At any point, the adversary forges a new, valid message/signature pair with respect to any oracle-generated verification key.
- (4) At any point during the experiment, a collision occurs in the hash function H (regardless of whether this is due to a direct action of the adversary, or whether this occurs during the course of the simulator's response to an oracle query).

It is immediate that event 2 occurs with negligible probability. It is also straightforward to show that events 1, 3, and 4 occur with negligible probability assuming the security of $(\text{Gen}, \text{Sign}, \text{Vrfy})$ as a one-time signature scheme and the security of CRHF as a collision-resistant hash family. We omit the obvious details.

Putting everything together, we see that $|\text{Adv}_{\mathcal{A}, P_0}(k) - \text{Adv}_{\mathcal{A}, P'_0}(k)|$ is negligible.

Experiment P_1 . In experiment P_1 , the simulator interacts with the adversary as in P'_0 except that the adversary's queries to the **Execute** oracle are handled differently: for each **Execute** query the values C and I are computed as $C := h^{r_1} \cdot g_1^{N+1}$ and $I := h^{r_2} \cdot g_1^{N+1}$; note that g_1^{N+1} is not a valid password. Furthermore, the session keys are computed as

$$\text{sk}_{\text{Client}}^i := \text{sk}_{\text{Server}}^j := A^{x_2} B^{y_2} (C / \text{pw}_{\text{Client}})^{z_2} D^{w_2} F^{x_1} G^{y_1} (I / \text{pw}_{\text{Client}})^{z_1} J^{w_1}. \quad (1)$$

The following bounds the effect this transformation can have on the adversary's advantage.

CLAIM 3.3. *Under the DDH assumption, $|\text{Adv}_{\mathcal{A}, P'_0}(k) - \text{Adv}_{\mathcal{A}, P_1}(k)|$ is negligible.*

The proof of the claim relies on the semantic security of the (modified) Cramer-Shoup encryption scheme (which is implied by its security against chosen-ciphertext attacks).⁸ We show that the simulator can use $\mathcal{A}$ as a subroutine to distinguish encryptions of the correct client passwords from encryptions of g_1^{N+1} . The simulator is given a public key $pk = \langle \mathbb{G}, g_1, g_2, h, c, d, H \rangle$ for an instance of the Cramer-Shoup scheme and may repeatedly query an encryption oracle $\mathcal{E}_{pk, \tilde{b}}(\cdot, \cdot, \cdot)$ where $\tilde{b}$ is a randomly-chosen bit (unknown to the simulator). The advantage of the simulator in attacking the encryption scheme is the absolute value of the difference between the probability the simulator outputs 1 when $\tilde{b} = 1$ and the probability the simulator outputs 1 when $\tilde{b} = 0$.

⁸ Earlier versions of this work [Katz 2002] show how to obtain a tighter concrete security reduction by relying directly on the DDH assumption. We have chosen to present a simpler proof here.

```

Execute(Client, i, Server, j) —
  if (Client ∉ Client or Server ∉ Server or usediClient or usedjServer)
    return ⊥
  usediClient := TRUE; usedjServer := TRUE
  PIDs := Client | Server; (VK, SK) ← Gen(1k)
  x1, x2, y1, y2, z1, z2, w1, w2 ← ℤq
  LABEL1 := PIDs | VK; ⟨A | B | C | D⟩ ← Epk, b̃(LABEL1, pwClient, g1N+1)
  msg1 := ⟨Client | VK | A | B | C | D⟩
  statusiClient,1 := ⟨NULL, Server, FALSE, FALSE⟩

  α := H(PIDs | VK | A | B | C); E := g1x2 g2y2 hz2 (cdα)w2
  LABEL2 := msg1 | Server | E; ⟨F | G | I | J⟩ ← Epk, b̃(LABEL2, pwClient, g1N+1)
  msg2 := ⟨Server | E | F | G | I | J⟩
  statusjServer,1 := ⟨NULL, Client, FALSE, FALSE⟩

  β := H(msg1 | Server | E | F | G | I)
  K := g1x1 g2y1 hz1 (cdβ)w1; Sig ← SignSK(msg1 | msg2 | K);
  msg3 := ⟨K | Sig⟩

  sidiClient := sidjServer := ⟨msg1 | msg2 | msg3⟩
  pidiClient := Server; pidjServer := Client
  acciClient := termiClient := accjServer := termjServer := TRUE
  skiClient := skjServer := Ax2 By2 (C / pwClient)z2 Dw2 Fx1 Gy1 (I / pwClient)z1 Jw1
  return msg1, msg2, msg3,
    statusiClient,1, statusjServer,1, statusiClient, statusjServer

```

FIG. 5. The modified Execute oracle for the proof of Claim 3.3.

The simulator begins by running the following modified initialization protocol:

```

Initialize'(1k, ℔, g1, g2, h, c, d, H) —
  (Client, Server) ← UserGen(1k)
  for each C ∈ Client
    pw'C ← {1, ..., N}
    pwC := g1pw'C
  for each S ∈ Server
    pws,C := pwC
  return Client, Server, ℔, g1, g2, h, c, d, H

```

The simulator responds to **Send**, **Reveal**, and **Test** queries as in experiments P'_0 and P_1 . However, it responds to **Execute** queries as shown in Figure 5. In words: each time the simulator responds to an **Execute** query it constructs the values A, B, C, D and F, G, I, J using its encryption oracle by querying this oracle using the appropriate label, and the correct password and g_1^{N+1} as its two “messages”. When $\mathcal{A}$ terminates, the simulator outputs 1 if and only if $\mathcal{A}$ succeeds.

It is easy to see that when $\tilde{b} = 0$ the actions of the **Execute** oracle are exactly as in experiment P'_0 , while if $\tilde{b} = 1$ the actions of the **Execute** oracle are exactly as in experiment P_1 . (In comparing Figures 3 and 5, it will be helpful to recall that when responding to an **Execute** query, pw_{Client} is always equal to $pw_{Server, Client}$ and furthermore sk_{Client}^i is always equal to sk_{Server}^j .) In particular, when $\tilde{b} = 0$ then

$C = h^{r_1} \cdot pw_{Client}$ and $I = h^{r_2} \cdot pw_{Client}$ (for $r_1 = \log_{g_1} A$ and $r_2 = \log_{g_1} F$ unknown to the simulator), while if $\tilde{b} = 1$ then $C = h^{r_1} \cdot g_1^{N+1}$ and $I = h^{r_2} \cdot g_1^{N+1}$. One can also verify that when $\tilde{b} = 0$ the session keys are computed as in experiment P'_0 , while if $\tilde{b} = 1$ the session keys are computed as in experiment P_1 .

The simulator's advantage in attacking the Cramer-Shoup scheme is therefore

$$|\Pr_{\mathcal{A}, P'_0}[\text{Succ}] - \Pr_{\mathcal{A}, P_1}[\text{Succ}]|.$$

The claim follows since this is negligible under the DDH assumption.

Experiment P_2 . In experiment P_2 , the simulator interacts with the adversary as in P_1 except that during queries $\text{Execute}(Client, i, Server, j)$ the session key sk_{Client}^i is chosen uniformly at random from $\mathbb{G}$; session key sk_{Server}^j is set equal to sk_{Client}^i .

CLAIM 3.4. $\text{Adv}_{\mathcal{A}, P_1}(k) = \text{Adv}_{\mathcal{A}, P_2}(k)$.

We will show that the distribution on the view of the adversary is identical in experiments P_1 and P_2 . For any particular Execute query made by the adversary in experiment P_1 , we may write $C = h^{r'_1} \cdot pw_{Client}$ with $r'_1 \neq r_1$ (recall from Figure 3 that $r_1 \stackrel{\text{def}}{=} \log_{g_1} A = \log_{g_2} B$). Now, for any $\mu, v \in \mathbb{G}$ and fixing the random choices for the remainder of experiment P_1 , the probability over choice of x_2, y_2, z_2, w_2 that $E = \mu$ and $sk_{Client}^i = v$ is exactly the probability that

$$\log_{g_1} \mu = x_2 + y_2 \cdot \log_{g_1} g_2 + z_2 \cdot \log_{g_1} h + w_2 \cdot \log_{g_1} (cd^\alpha) \quad (2)$$

and

$$\begin{aligned} & \log_{g_1} v - \log_{g_1} (F^{x_1} G^{y_1} (I / pw_{Client})^{z_1} J^{w_1}) \\ &= x_2 \cdot r_1 + y_2 \cdot r_1 \log_{g_1} g_2 + z_2 \cdot r'_1 \log_{g_1} h + w_2 \cdot r_1 \log_{g_1} (cd^\alpha), \end{aligned} \quad (3)$$

where we use the fact that g_1 is a generator. Viewing Eqs. (2) and (3) as equations over $\mathbb{Z}_q$ in the variables x_2, y_2, z_2, w_2 , we see that they are linearly independent and not identically zero since $r'_1 \neq r_1$ (here, we use the fact that h is a generator and hence $\log_{g_1} h \neq 0$). Thus, the desired probability is $1/q^2$. In other words, the value of sk_{Client}^i is independent of the value of E and hence independent of the remainder of experiment P_1 . Thus, the adversary's view in experiment P_1 is distributed identically to the adversary's view in experiment P_2 . The claim follows.

In experiment P_2 , the adversary's probability of correctly guessing the bit b used by the Test oracle is exactly $1/2$ if the Test query is made to a fresh instance that was activated using an Execute query. This is so because session keys for such instances in P_2 are chosen at random from $\mathbb{G}$, and hence there is no way to distinguish whether the Test oracle outputs a random session key or the "actual" session key (which is just a random element, anyway). Hence, the remainder of the proof concentrates on instances that are invoked via Send queries (which is the more difficult case to consider).

Experiment P_3 . In experiment P_3 , the simulator first runs the following modified initialization procedure and stores the values $\kappa, \chi_1, \chi_2, \xi_1, \xi_2$ for future use.

```

Initialize( $1^k$ ) —
   $\mathbb{G} \leftarrow \mathcal{G}(1^k); g_1, g_2 \leftarrow \tilde{\mathbb{G}}$ 
   $\kappa \leftarrow \mathbb{Z}_q^*$ 
   $(\chi_1, \chi_2), (\xi_1, \xi_2) \leftarrow \{(x, y) \in \mathbb{Z}_q \times \mathbb{Z}_q \mid g_1^x g_2^y \neq 1\}$ 
   $h := g_1^\kappa; c := g_1^{\chi_1} g_2^{\chi_2}; d := g_1^{\xi_1} g_2^{\xi_2}$ 
   $H \leftarrow \text{CRHF}(1^k)$ 
   $(\text{Client}, \text{Server}) \leftarrow \text{UserGen}(1^k)$ 
  for each  $C \in \text{Client}$ 
     $pw'_C \leftarrow \{1, \dots, N\}$ 
     $pw_C := g_1^{pw'_C}$ 
  for each  $S \in \text{Server}$ 
     $pw_{S,C} := pw_C$ 
  return  $\text{Client}, \text{Server}, \mathbb{G}, g_1, g_2, h, c, d, H$ 

```

The simulator's handling of **Send** oracle queries in P_3 will also change, but before describing this we introduce some terminology. For a query $\text{Send}_1(\text{Server}, j, \text{msg}_1)$ where msg_1 is adversarially-generated, let $\text{msg}_1 = \langle \text{Client} | \text{VK} | A | B | C | D \rangle$ and $\alpha = H(\text{PIDs} | \text{VK} | A | B | C)$. If either $A^{\chi_1 + \alpha \xi_1} B^{\chi_2 + \alpha \xi_2} \neq D$ or $C / pw_{\text{Client}} \neq A^\kappa$, then msg_1 is said to be *invalid*. Otherwise, msg_1 is said to be *valid*. Similarly, for a query $\text{Send}_2(\text{Client}, i, \text{msg}_2)$ where msg_2 is adversarially-generated, let $\text{msg}_2 = \langle \text{Server} | E | F | G | I | J \rangle$ and β be as defined in Figure 4. If either $F^{\chi_1 + \beta \xi_1} G^{\chi_2 + \beta \xi_2} \neq J$ or $I / pw_{\text{Client}} \neq F^\kappa$ then msg_2 is said to be *invalid*. Otherwise, msg_2 is said to be *valid*. Informally, valid messages are encryptions of the correct password while invalid messages are not. Note that the simulator can efficiently determine whether any given adversarially generated message is valid because it knows $\kappa, \chi_1, \chi_2, \xi_1, \xi_2$.

Given this terminology, we continue with our description of experiment P_3 . When the adversary makes oracle query $\text{Send}_2(\text{Client}, i, \text{msg}_2)$, the simulator examines msg_2 . If msg_2 is adversarially-generated and valid, the query is answered as in P_2 except that $\text{sk}_{\text{Client}}^i$ is assigned⁹ the special value ∇ . In any other case (i.e., msg_2 is oracle-generated, or adversarially-generated but invalid), the query is answered exactly as in experiment P_2 .

When the adversary queries $\text{Send}_3(\text{Server}, j, \text{msg}_3)$, the simulator examines $\text{msg}_1 = \text{first-msg-in}$ for instance Π_{Server}^j .¹⁰ If msg_1 is adversarially-generated and valid, the query is answered as in experiment P_2 except that $\text{sk}_{\text{Server}}^j$ is assigned the special value ∇ . In any other case (i.e., msg_1 is oracle-generated, or adversarially-generated but invalid), the query is answered exactly as in experiment P_2 .

Finally, the definition of the adversary's success in P_3 is changed. If the adversary ever queries $\text{Reveal}(U, i)$ or $\text{Test}(U, i)$ where $\text{sk}_U^i = \nabla$, the simulator halts and the adversary succeeds. Otherwise, the adversary's success is determined as in experiment P_2 .

⁹Here and in the remainder of the proof, it is understood that when we say a certain variable is assigned some value this is conditioned on that variable being assigned a value at all (and, e.g., the oracle query does not return $\perp$ for some other reason, nor is the experiment aborted).

¹⁰If the adversary had not previously made a Send_1 query for this instance, the simulator responds with $\perp$ as in previous experiments. We do not explicitly mention this in the rest of the proof.

CLAIM 3.5. $\text{Adv}_{\mathcal{A}, P_2}(k) \leq \text{Adv}_{\mathcal{A}, P_3}(k)$.

The distribution of the public parameters is identical in experiments P_2 and P_3 . Furthermore, the distributions on the adversary's view in experiments P_2 and P_3 are identical up to the point when the adversary queries $\text{Reveal}(U, i)$ or $\text{Test}(U, i)$ with $\text{sk}_U^i = \nabla$; if such a query is never made, the distributions on the view are identical. Since the adversary succeeds if such a query ever occurs, the claim follows.

Experiment P_4 . In experiment P_4 , we again modify the simulator's response to a Send_3 query. Upon receiving query $\text{Send}_3(\text{Server}, j, \langle K | \text{Sig} \rangle)$, the simulator examines $\text{msg}_1 = \text{first-msg-in}$ for instance Π_{Server}^j . If msg_1 is oracle-generated and a value is to be assigned to $\text{sk}_{\text{Server}}^j$, the simulator finds the unique pair Client, i such that $\text{sid}_{\text{Client}}^i = \text{sid}_{\text{Server}}^j$ (we comment on the existence of a unique such pair below) and sets $\text{sk}_{\text{Server}}^j := \text{sk}_{\text{Client}}^i$. In any other case, the query is answered as in experiment P_3 .

That there exists a unique pair Client, i with $\text{sid}_{\text{Client}}^i = \text{sid}_{\text{Server}}^j$ in the situation above follows from the fact that, assuming $\text{sk}_{\text{Server}}^j$ is to be assigned a value — and so in particular the simulation is not aborted — the following hold: (1) there is a unique pair Client, i such that instance Π_{Client}^i uses VK, where VK is the verification key contained in msg_1 (this is a consequence of msg_1 being oracle-generated, and the fact that the simulator aborts if an oracle-generated verification key is ever used twice); furthermore, (2) this pair satisfies $\text{sid}_{\text{Client}}^i = \text{sid}_{\text{Server}}^j$ (this is a consequence of the fact that the simulator aborts if a signature forgery occurs).

Since $\text{sid}_{\text{Server}}^j = \text{sid}_{\text{Client}}^i$ implies $\text{sk}_{\text{Server}}^j = \text{sk}_{\text{Client}}^i$ in P_3 , it is immediate that $\text{Adv}_{\mathcal{A}, P_4}(k) = \text{Adv}_{\mathcal{A}, P_3}(k)$.

Experiment P_5 . In experiment P_5 , we again modify the simulator's response to a Send_3 query. Upon receiving query $\text{Send}_3(\text{Server}, j, \langle K | \text{Sig} \rangle)$, the simulator examines $\text{msg}_1 = \text{first-msg-in}$ for the same instance Π_{Server}^j . If msg_1 is adversarially-generated and invalid, the session key $\text{sk}_{\text{Server}}^j$ is assigned a value randomly chosen from $\mathbb{G}$. In all other cases, the query is answered as in experiment P_4 .

CLAIM 3.6. $\text{Adv}_{\mathcal{A}, P_5}(k) = \text{Adv}_{\mathcal{A}, P_4}(k)$.

We prove the claim by showing that the distributions on the adversary's view in the two experiments are identical. For a given query $\text{Send}_3(\text{Server}, j, \langle K | \text{Sig} \rangle)$ where $\text{msg}_1 = \text{first-msg-in} = \langle \text{Client} | \text{VK} | A | B | C | D \rangle$ is adversarially generated and invalid, let $\text{msg}_2 = \text{first-msg-out} = \langle \text{Server} | E | F | G | I | J \rangle$ and $\alpha = H(\text{PIDS} | \text{VK} | A | B | C)$. Since msg_1 is invalid, either $A^{\chi_1 + \alpha \xi_1} B^{\chi_2 + \alpha \xi_2} \neq D$ or else $C/pw_{\text{Client}} \neq A^\kappa$ (or possibly both). For any $\mu, \nu \in \mathbb{G}$ and fixing the randomness used in the rest of experiment P_4 , the probability over choice of x, y, z, w that $E = \mu$ and $\text{sk}_{\text{Server}}^j = \nu$ is exactly the probability that

$$\log_{g_1} \mu = x + y \cdot \log_{g_1} g_2 + z \cdot \log_{g_1} h + w \cdot \log_{g_1} (cd^\alpha) \quad (4)$$

and

$$\begin{aligned} & \log_{g_1} \nu - r \log_{g_1} K \\ &= x \cdot \log_{g_1} A + y \cdot \log_{g_1} B + z \cdot \log_{g_1} (C/pw_{\text{Client}}) + w \cdot \log_{g_1} D, \end{aligned} \quad (5)$$

using the fact that g_1 is a generator. Consider two cases depending on the value of $\log_{g_1} A$. If $\log_{g_1} A = 0$, then since msg_1 is invalid at least one of the values $\log_{g_1} B$, $\log_{g_1}(C/pw_{\text{Client}})$, or $\log_{g_1} D$ is not equal to 0. One can verify that Eqs. (4) and (5), viewed as equations over $\mathbb{Z}_q$ in the variables x, y, z, w , are linearly independent and not identically zero. If $\log_{g_1} A \neq 0$, it can be similarly verified that Eqs. (4) and (5) are linearly independent and not identically zero. In either case, then, the desired probability is $1/q^2$. Thus, the value of $\text{sk}_{\text{Server}}^j$ is independent of the value of E and hence independent of the remainder of the experiment.

Before describing the next experiment, let us first summarize where things stand in experiment P_5 . In responding to a query $\text{Send}_3(S, j, \text{msg}_3)$ in that experiment, the simulator does not need to use the value r (stored as part of the internal state for instance Π_S^j) to compute the value of the session key. In particular, if msg_1 was the initial message sent to this instance then: (1) If msg_1 is adversarially generated and valid, the simulator sets sk_S^j equal to ∇ as described in experiment P_3 ; (2) if msg_1 is oracle-generated, the simulator sets sk_S^j equal to the session key for the partnered client instance as described in experiment P_4 ; (3) if msg_1 is adversarially-generated and invalid, the simulator chooses sk_S^j at random as described in experiment P_5 .

Experiment P_6 . In experiment P_6 , we modify the simulator's response to Send_1 queries. Now, I is computed as $h^r g_1^{N+1}$, where the dictionary of legal passwords is $\{1, \dots, N\}$; note that g_1^{N+1} is not a valid password since $N < q$.

CLAIM 3.7. *Under the DDH assumption, $|\text{Adv}_{\mathcal{A}, P_5}(k) - \text{Adv}_{\mathcal{A}, P_6}(k)|$ is negligible.*

A proof of the claim relies on the security of the (modified) Cramer-Shoup encryption scheme against chosen-ciphertext attacks; cf. Section 2.4. We show that the simulator can use $\mathcal{A}$ as a subroutine in order to distinguish encryptions of the correct client password(s) from encryptions of g_1^{N+1} . The simulator is given a public key $pk = \langle \mathbb{G}, g_1, g_2, h, c, d, H \rangle$ for an instance of the Cramer-Shoup encryption scheme and may repeatedly query an encryption oracle $\mathcal{E}_{pk, \tilde{b}}(\cdot, \cdot, \cdot)$ where $\tilde{b}$ is a randomly chosen bit (unknown to the simulator). The simulator may also repeatedly query a decryption oracle $\mathcal{D}_{sk}(\cdot, \cdot)$ subject to the restriction stated in Definition 6. By definition, the advantage of the simulator in attacking the encryption scheme is the absolute value of the difference between the probability the simulator outputs 1 when $\tilde{b} = 1$ and the probability the simulator outputs 1 when $\tilde{b} = 0$.

The simulator begins by running the following modified initialization protocol:

```

Initialize'(1k,  $\mathbb{G}, g_1, g_2, h, c, d, H$ ) —
  (Client, Server)  $\leftarrow$  UserGen(1k)
  for each  $C \in \text{Client}$ 
     $pw'_C \leftarrow \{1, \dots, N\}$ 
     $pw_C := g_1^{pw'_C}$ 
  for each  $S \in \text{Server}$ 
     $pw_{s,C} := pw_C$ 
  return Client, Server,  $\mathbb{G}, g_1, g_2, h, c, d, H$ 

```

The simulator responds to Send_0 , Execute , Reveal , and Test oracle queries as in experiments P_5 and P_6 . The simulator responds to Send_1 , Send_2 , and Send_3

queries as shown in Figure 6. In words:

- In responding to a Send_1 query the simulator no longer computes the values F, G, I, J on its own. Instead, it queries its encryption oracle using (1) the appropriate label, and (2) the correct password and g_1^{N+1} as its two “messages.” It then uses the ciphertext $\langle F|G|I|J \rangle$ returned by this oracle to construct msg-out .
- In responding to a Send_3 query, the simulator first checks whether $\text{msg}_1 = \text{first-msg-in}$ is oracle- or adversarially-generated. If msg_1 is oracle-generated, the session key is computed by finding the partnered client instance (as discussed above). If msg_1 is adversarially-generated, the simulator determines whether msg_1 is valid or invalid using its decryption oracle and then sets the session key appropriately.
- In responding to a Send_2 query, the simulator uses its decryption oracle (as needed) to determine whether an adversarially-generated incoming message is valid or invalid.

Note that the simulator never need submit to its decryption oracle an “illegal” query (i.e., a ciphertext that it previously received from its encryption oracle) since the simulator never needs to check whether oracle-generated messages are valid or invalid. Finally, the simulator outputs 1 if and only if $\mathcal{A}$ succeeds.

Examination of Figure 6 shows that the Send_2 and Send_3 oracles behave exactly the same as in both experiments P_5 and P_6 . On the other hand, when $\tilde{b} = 0$ the actions of the Send_1 oracle are as in experiment P_5 , while if $\tilde{b} = 1$ then the actions of the Send_1 oracle are as in experiment P_6 . The simulator’s advantage in attacking the Cramer-Shoup encryption scheme is therefore exactly $|\Pr_{\mathcal{A}, P_5}[\text{Succ}] - \Pr_{\mathcal{A}, P_6}[\text{Succ}]|$. The claim follows since this advantage is negligible under the DDH assumption, since the Cramer-Shoup scheme is secure given this assumption (cf. Section 2.4).

Experiment P_7 . In experiment P_7 , queries to the Send_2 oracle are handled differently. In response to a query $\text{Send}_2(\text{Client}, i, \text{msg}_2)$, if msg_2 is either oracle-generated or adversarially generated but invalid, the session key $\text{sk}_{\text{Client}}^i$ is assigned a value chosen randomly from $\mathbb{G}$.

CLAIM 3.8. $\text{Adv}_{\mathcal{A}, P_7}(k) = \text{Adv}_{\mathcal{A}, P_6}(k)$.

We prove the claim by showing that the distributions on the adversary’s view are identical in the two experiments. Consider a particular query $\text{Send}_2(\text{Client}, i, \text{msg}_2)$ in P_6 . Regardless of whether msg_2 is oracle-generated or adversarially-generated but invalid, it is the case that either $F^{X_1 + \beta \xi_1} G^{X_2 + \beta \xi_2} \neq J$ or $I / pw_{\text{Client}} \neq F^\kappa$. A proof similar to that used in proving Claim 3.6 then shows that $\text{sk}_{\text{Client}}^i$ in experiment P_6 is uniformly distributed in $\mathbb{G}$, independent of the rest of the experiment.

Experiment P_8 . In experiment P_8 , we modify the simulator’s response to Send_0 queries. Now, C is computed as $C := h^r g_1^{N+1}$; note that g_1^{N+1} is not a valid password. (This modification parallels the change made in going from experiment P_5 to experiment P_6 .)

CLAIM 3.9. *Under the DDH assumption, $|\text{Adv}_{\mathcal{A}, P_8}(k) - \text{Adv}_{\mathcal{A}, P_7}(k)|$ is negligible.*

```

Send1(Server, j, ⟨Client|VK|A|B|C|D⟩) —
  if (Server ∉ Server or usedServerj) return ⊥
  usedServerj := TRUE; PIDs := Client|Server
  if A, B, C, D ∉ G or Client ∉ Client
    statusServerj := ⟨NULL, NULL, FALSE, TRUE⟩; return statusServerj
  x, y, z, w ← Zq; α := H(PIDs|VK|A|B|C)
  E := g1xg2yhz(cdα)w; LABEL := msg-in|Server|E
  ⟨F|G|I|J⟩ ← Epk, b(LABEL, pwServer, Client, g1N+1)
  msg-out := ⟨Server|E|F|G|I|J⟩
  statusServerj := ⟨NULL, Client, FALSE, FALSE⟩
  stateServerj := ⟨msg-in, x, y, z, w, msg-out, pwServer, Client⟩
  return msg-out, statusServerj

Send2(Client, i, ⟨Server|E|F|G|I|J⟩) —
  if (Client ∉ Client or ¬usedClienti or termClienti) return ⊥
  if Server ≠ pidClienti or E, F, G, I, J ∉ G
    statusClienti := ⟨NULL, NULL, FALSE, TRUE⟩; return statusClienti
  ⟨SK, r, first-msg-out⟩ := stateClienti
  x, y, z, w ← Zq; β := H(first-msg-out|Server|E|F|G|I)
  K := g1xg2yhz(cdβ)w; Sig ← SignSK(first-msg-out|msg-in|K);
  msg-out := ⟨K|Sig⟩
  sidClienti := ⟨first-msg-out|msg-in|msg-out⟩
  accClienti := termClienti := TRUE
  LABEL := first-msg-out|Server|E
  if msg-in is adversarially-generated and Dsk(LABEL, ⟨F|G|I|J⟩) = pwClient
    skClienti := ∇
  else
    skClienti := ErFxGy(I/pwClient)zJw
  return msg-out, statusClienti

Send3(Server, j, ⟨K|Sig⟩) —
  if (Server ∉ Server or ¬usedServerj or termServerj) return ⊥
  ⟨first-msg-in, x, y, z, w, first-msg-out, pw⟩ := stateServerj
  ⟨Client|VK|A|B|C|D⟩ := first-msg-in
  if K ∉ G or VrfyVK(first-msg-in|first-msg-out|K, Sig) ≠ 1
    statusServerj := ⟨NULL, NULL, FALSE, TRUE⟩; return statusServerj
  sidServerj := ⟨first-msg-in|first-msg-out|msg-in⟩
  accServerj := termServerj := TRUE
  if first-msg-in is oracle-generated
    find the unique i such that sidClienti = sidServerj
    skServerj := skClienti
  else
    LABEL := Client|Server|VK
    pw' := Dsk(LABEL, ⟨A|B|C|D⟩)
    if pw' = pw then skServerj := ∇
    if pw' ≠ pw then skServerj ← G
  return statusServerj

```

FIG. 6. The modified Send oracles for the proof of Claim 3.7.

The proof exactly follows that of Claim 3.7. In particular, note that in responding to a query $\text{Send}_2(C, i, \text{msg}_2)$ in experiment P_7 , the simulator never requires the value r (stored as part of the internal state for instance Π_C^i) to compute a session key: if msg_2 is oracle-generated, the session key is assigned a randomly chosen value; if msg_2 is adversarially generated and invalid (which can be verified using the decryption oracle), the session key is also assigned a randomly chosen value; finally, if msg_2 is adversarially generated and valid (which can again be verified using the decryption oracle), the session key is assigned ∇ . Oracle queries to Send_3 can also be answered by the simulator, using its decryption oracle as necessary. Finally, there is never a need for the simulator to make an “illegal” query its decryption oracle. The claim follows:

The adversary’s view in experiment P_8 is independent of the passwords chosen by the simulator until one of the following occur:

- The adversary queries $\text{Reveal}(Client, i)$ or $\text{Test}(Client, i)$, and had previously queried $\text{Send}_2(Client, i, \text{msg}_2)$ for adversarially generated and valid msg_2 .
- The adversary queries $\text{Reveal}(Server, j)$ or $\text{Test}(Server, j)$, and had previously queried $\text{Send}_1(Server, j, \text{msg}_1)$ for adversarially generated and valid msg_1 .

The probability that either of these events occur is at most $Q(k)/N$, where $Q(k)$ is the number of on-line attacks made by $\mathcal{A}$. If neither of the above events occur, then the adversary succeeds only if it can guess the value of b used by the Test oracle. However, if neither of the above events occur and the adversary queries $\text{Test}(U, i)$ where Π_U^i is fresh and $\text{acc}_U^i = \text{TRUE}$, then sk_U^i is randomly-distributed in $\mathbb{G}$ independent of $\mathcal{A}$ ’s view; the probability of correctly guessing b in this case is therefore exactly $1/2$.

The preceding discussion implies that

$$\Pr_{\mathcal{A}, P_8}[\text{Succ}] \leq Q(k)/N + \frac{1}{2} \cdot (1 - \frac{Q(k)}{N})$$

and thus the adversary’s advantage in experiment P_8 is at most $Q(k)/N$. The sequence of claims proven above show that

$$\text{Adv}_{\mathcal{A}, P_0}(k) \leq \text{Adv}_{\mathcal{A}, P_8}(k) + \varepsilon(k)$$

for some negligible function $\varepsilon(\cdot)$ and therefore the adversary’s advantage in P_0 (i.e., the original protocol) is at most $Q(k)/N$ plus some negligible quantity. This completes the proof of the theorem. $\square$

3.3. PROOF OF FORWARD SECRECY. We first demonstrate a potential attack on the protocol (in the stronger adversarial model of Section 2.3) if it is modified in a seemingly innocuous way. Then, we show a proof that the protocol as formally defined does indeed achieve forward secrecy.

Consider the following change to the protocol as described in the previous section: when responding to a Send_1 query, a server no longer stores $\text{pw}_{S,C}$ as part of its state information (cf. Figure 4). Now, when a server responds to a Send_3 query it simply sets $\text{pw} = \text{pw}_{S,C}$. In other words, instead of using the password stored as part of its (temporary, volatile) state information, the server simply uses the value of the client’s password as stored in its long-term memory (e.g., the password file).

We now show that this simple change introduces a vulnerability in the attack model described in Section 2.3. Fix a client $Client$. The attack begins by having the adversary impersonate $Client$ to a server S using an arbitrary password, say

$pw = g_1$. That is, the adversary generates VK , chooses a random r_1 , computes $A = g_1^{r_1}$, $B = g_2^{r_1}$, $C = h_{r_1} \cdot g_1$, and $D = (cd^\alpha)^{r_1}$ (for α computed in the appropriate way), and sends $\langle Client|VK|A|B|C|D \rangle$ to server S . The server will respond with a message $\langle S|E|F|G|I|J \rangle$ (which is computed using password $pw_{S,Client} = pw_{Client}$). Next, the adversary changes the password stored at S for $Client$ (i.e., $pw_{S,Client}$) to the value g_1 using oracle call $Corrupt_2(S, Client, g_1)$. Finally, the adversary computes the final message of the protocol just as a legitimate client would by choosing x_1, y_1, z_1, w_1 , computing $K = g_1^{x_1} g_2^{y_1} h^{z_1} (cd^\beta)^{w_1}$ (for β computed in the appropriate way), and sending $\langle K|Sig \rangle$. Finally, the adversary obtains sk_S for this instance of the server using a **Reveal** query.

We claim that the adversary can determine pw_{Client} — the original password of the client — and hence successfully impersonate $Client$ to a *different* server $S' \neq S$. To see this, note that when S computes the session key after receiving the final message of the protocol it uses $pw_{S,Client} = g_1$ (which, recall, is different from the password the server used to compute the second message of the protocol). Thus,

$$\begin{aligned} sk_S &= A^{x_2} B^{y_2} (C/g_1)^{z_2} D^{w_2} K^{r_2} = g_1^{r_1 x_2} g_2^{r_1 y_2} h^{r_1 z_2} (cd^\alpha)^{r_1 w_2} K^{r_2} \\ &= E^{r_1} K^{r_2} \end{aligned}$$

(where x_2, y_2, z_2, w_2, r_2 are values used by the server which are unknown to the adversary). Since the adversary can compute E^{r_1} itself, the adversary can use sk_S to determine the value $K^{r_2} = sk_S / E^{r_1}$. By exhaustively searching the password dictionary, the adversary can identify pw_{Client} as the unique value for which:

$$F^{x_1} G^{y_1} (I / pw_{Client})^{z_1} J^{w_1} = K^{r_2}.$$

The protocol as formally specified in the previous section is not vulnerable to this attack since it ensures that a server-instance uses the same password throughout its execution by storing the client's password as part of the state information for that instance. Interestingly, this means that it is possible for two server-instances active at the same time to be using different passwords for a given client (the password being used will be determined by the value of $pw_{S,C}$ at the time the initial **Send**₁ query for each instance was made).

We now provide a rigorous proof that the protocol as specified achieves forward secrecy.

THEOREM 3.10. *Under the same assumptions as in Theorem 3.1, the protocol of Figure 1 achieves forward secrecy.*

PROOF. A significant portion of the proof is similar to the proof of Theorem 3.1, and so we will focus only on the differences here. As in the previous proof, we imagine a simulator that runs the protocol for any adversary $\mathcal{A}$ (the simulator must now also simulate the **Corrupt** oracles for $\mathcal{A}$ but this does not present any problems). When the adversary completes its execution and outputs a bit b' , the simulator can tell whether the adversary succeeds by checking whether (1) a single **Test** query was made on instance Π_U^i ; (2) acc_U^i was TRUE at the time of the **Test** query; (3) instance Π_U^i is fresh (according to the definition of freshness appropriate for this setting; see Section 2.3); and (4) $b' = b$. Success of the adversary is denoted by event **fsSucc**, and for any game P we let $fsAdv_{\mathcal{A},P}(k) \stackrel{\text{def}}{=} 2 \cdot \Pr_{\mathcal{A},P}[\text{fsSucc}] - 1$. We refer to the real execution of the experiment as P_0 .

Experiment P'_0 . We define experiment P'_0 exactly as in the previous proof; as there, it is clear that $|\text{fsAdv}_{\mathcal{A}, P_0}(k) - \text{fsAdv}_{\mathcal{A}, P'_0}(k)|$ is negligible.

Experiment P_1 . In experiment P_1 , the simulator interacts with the adversary as in P'_0 except that the adversary's queries to the **Execute** oracle are handled differently. In response to the adversary's oracle query **Execute**($Client, i, Server, j$), the simulator checks whether $pw_{Client} = pw_{Server, Client}$. If so, the values C, I are computed as $C := h^{r_1} \cdot g_1^{N+1}$ and $I := h^{r_2} \cdot g_1^{N+1}$ (as before, the key point is that g_1^{N+1} is not a valid password), and the session keys are computed as

$$\text{sk}_{Client}^i := \text{sk}_{Server}^j := A^{x_2} B^{y_2} (C / pw_{Client})^{z_2} D^{w_2} F^{x_1} G^{y_1} (I / pw_{Client})^{z_1} J^{w_1}.$$

On the other hand, if $pw_{Client} \neq pw_{Server, Client}$, then the values C, I are computed as $C := h^{r_1} \cdot g_1^{N+1}$ and $I := h^{r_2} \cdot g_1^{N+2}$, and the session keys are computed as

$$\begin{aligned} \text{sk}_{Client}^i &:= A^{x_2} B^{y_2} (C / pw_{Client})^{z_2} D^{w_2} F^{x_1} G^{y_1} (I / pw_{Client})^{z_1} J^{w_1} \\ \text{sk}_{Server}^j &:= A^{x_2} B^{y_2} (C / pw_{Server, Client})^{z_2} D^{w_2} F^{x_1} G^{y_1} (I / pw_{Server, Client})^{z_1} J^{w_1}. \end{aligned}$$

(We implicitly assume here that $N + 2 \leq q$ and so neither g_1^{N+1} nor g_1^{N+2} are valid passwords.) Using essentially the same proof as in Claim 3.3, semantic security of the (modified) Cramer-Shoup encryption scheme implies that $|\text{fsAdv}_{\mathcal{A}, P'_0}(k) - \text{fsAdv}_{\mathcal{A}, P_1}(k)|$ is negligible.

Experiment P_2 . In experiment P_2 , the simulator interacts with the adversary as in P_1 except that during queries **Execute**($Client, i, Server, j$) the simulator proceeds as follows: if $pw_{Client} = pw_{Server, Client}$ then sk_{Client}^i is chosen uniformly at random from $\mathbb{G}$ and sk_{Server}^j is set equal to sk_{Client}^i . Otherwise, both sk_{Client}^i and sk_{Server}^j are chosen independently and uniformly from $\mathbb{G}$.

CLAIM 3.11. $\text{fsAdv}_{\mathcal{A}, P_1}(k) = \text{fsAdv}_{\mathcal{A}, P_2}(k)$.

The claim follows since the view of the adversary is distributed identically in both experiments. The case of $pw_{Client} = pw_{Server, Client}$ exactly follows the proof of Claim 3.4. Consider an invocation of the **Execute** oracle when $pw_{Client} \neq pw_{Server, Client}$ (for brevity, denote pw_{Client} by pw_c and $pw_{Server, Client}$ by $pw_{s,c}$). We may write

$$C = h^{r_1} \cdot g_1^{N+1} = h^{r'_1} \cdot pw_c = h^{r''_1} \cdot pw_{s,c}$$

for some r'_1, r''_1 , and

$$I = h^{r_2} \cdot g_1^{N+2} = h^{r'_2} \cdot pw_c = h^{r''_2} \cdot pw_{s,c}$$

for some r'_2, r''_2 . Note that none of r_1, r'_1, r''_1 are equal, and similarly for r_2, r'_2, r''_2 . Furthermore, we have $r''_1 - r'_1 = r''_2 - r'_2$ but $r'_1 - r_1 \neq r'_2 - r_2$.

Now, for any $\mu_1, \mu_2, \nu_1, \nu_2 \in \mathbb{G}$ and fixing the random choices for the remainder of experiment P_1 , the probability over choice of $x_1, y_1, z_1, w_1, x_2, y_2, z_2, w_2$ that

$E = \mu_1, K = \mu_2, \text{sk}_{\text{Client}}^i = v_1$, and $\text{sk}_{\text{Server}}^j = v_2$ is exactly the probability that

$$\log_{g_1} \mu_1 = x_2 + y_2 \cdot \log_{g_1} g_2 + z_2 \cdot \log_{g_1} h + w_2 \cdot \log_{g_1} (cd^\alpha) \quad (6)$$

$$\log_{g_1} \mu_2 = x_1 + y_1 \cdot \log_{g_1} g_2 + z_1 \cdot \log_{g_1} h + w_1 \cdot \log_{g_1} (cd^\beta) \quad (7)$$

$$\begin{aligned} \log_{g_1} v_1 = & x_1 \cdot r_2 + y_1 \cdot r_2 \log_{g_1} g_2 + z_1 \cdot r_2' \log_{g_1} h + w_1 \cdot r_2 \log_{g_1} (cd^\beta) \\ & + x_2 \cdot r_1 + y_2 \cdot r_1 \log_{g_1} g_2 + z_2 \cdot r_1' \log_{g_1} h + w_2 \cdot r_1 \log_{g_1} (cd^\alpha) \end{aligned} \quad (8)$$

$$\begin{aligned} \log_{g_1} v_2 = & x_1 \cdot r_2 + y_1 \cdot r_2 \log_{g_1} g_2 + z_1 \cdot r_2'' \log_{g_1} h + w_1 \cdot r_2 \log_{g_1} (cd^\beta) \\ & + x_2 \cdot r_1 + y_2 \cdot r_1 \log_{g_1} g_2 + z_2 \cdot r_1'' \log_{g_1} h + w_2 \cdot r_1 \log_{g_1} (cd^\alpha). \end{aligned} \quad (9)$$

It may be verified that Eqs. (6)–(9) are linearly independent and not identically zero, and so the values $E, K, \text{sk}_{\text{Client}}^i$, and $\text{sk}_{\text{Server}}^j$ are independently and uniformly distributed, independent of the rest of the experiment. The claim follows.

Before continuing, we introduce some new terminology. First, we say that instance Π_U^i is *associated with* U' if either $U = U'$ or $\text{pid}_U^i = U'$. More interestingly, we call a query $\text{Send}_0(C, *, *)$ or $\text{Send}_2(C, *, *)$ *corrupted* if the adversary had previously queried $\text{Corrupt}_1(C)$. Similarly, we call a query $\text{Send}_1(S, i, *)$ (with $\text{pid}_S^i = C$) *corrupted* if the adversary had previously queried either $\text{Corrupt}_1(C)$ or $\text{Corrupt}_2(S, C, *)$. We call a query $\text{Send}_3(S, i, *)$ *corrupted* exactly when the corresponding query $\text{Send}_1(S, i, *)$ (i.e., the Send_1 query for the same instance) is corrupted. Note that for any corrupted query $\text{Send}_*(U, i, *)$ the adversary “knows” the password being used by Π_U^i at the time this query is made, and furthermore the instance Π_U^i is not fresh (as per the definition given in Section 2.3).

The experiments we now introduce exactly parallel those introduced in the previous proof, with the main differences being that we only change the simulator’s actions in response to noncorrupted Send queries.

Experiment P_3 . In experiment P_3 , the simulator runs the modified initialization procedure as in the proof of Theorem 3.1, with the values $\kappa, \chi_1, \chi_2, \xi_1, \xi_2$ stored as before. In a way analogous to (but slightly different from) the previous proof, we define a msg_1 or msg_2 to be *oracle-generated* if it is output as a result of an Execute query or in response to a *noncorrupted* Send_0 or Send_1 query. Otherwise a message is called *adversarially generated*. We also define *valid* and *invalid* exactly as in the previous proof. We stress, however, that to determine whether a message msg_1 submitted to the Send_1 oracle is valid/invalid, the value of $\text{pw}_{\text{Server}, \text{Client}}$ at the time of the Send_1 query is used (even if this value is changed at some later point in the experiment via a Corrupt_2 query).

We now continue with our description of experiment P_3 . When the adversary makes a noncorrupted oracle query $\text{Send}_2(\text{Client}, i, \text{msg}_2)$ and msg_2 is adversarially-generated and valid, the query is answered as in experiment P_2 but the simulator stores ∇ as the “session key” for this instance. In any other case (i.e., msg_2 is oracle-generated, or adversarially-generated but invalid), the query is exactly answered as in experiment P_2 .

When the adversary makes noncorrupted oracle query $\text{Send}_3(\text{Server}, j, \text{msg}_3)$, the simulator examines $\text{msg}_1 = \text{first-msg-in}$ for instance Π_{Server}^j . If msg_1 is adversarially generated and valid, the query is answered as in experiment P_2 but the simulator stores ∇ as the “session key” for this instance. In any other case (i.e., msg_1

is oracle-generated, or adversarially generated but invalid), the query is answered exactly as in experiment P_2 .

Finally, the definition of the adversary's success is changed. If the adversary queries $\text{Test}(U, i)$ or $\text{Reveal}(U, i)$ and $\text{sk}_U^i = \nabla$, the simulator halts and the adversary succeeds. The adversary also succeeds if it queries $\text{Corrupt}_1(C)$ and there exists any instance associated with C which has session key ∇ . Otherwise, the adversary succeeds as in experiment P_2 .

Since there are more ways for the adversary to succeed in P_3 and the experiment is otherwise identical to P_2 from the adversary's point of view, it follows immediately (as in Claim 3.5) that $\text{fsAdv}_{\mathcal{A}, P_2}(k) \leq \text{fsAdv}_{\mathcal{A}, P_3}(k)$.

Experiment P_4 . In experiment P_4 , we again modify the simulator's response to a noncorrupted Send_3 query made by the adversary. Upon receiving a noncorrupted query $\text{Send}_3(\text{Server}, j, \langle K | \text{Sig} \rangle)$, the simulator examines $\text{msg}_1 = \text{first-msg-in}$ for instance Π_{Server}^j . If msg_1 is oracle-generated and a value is to be assigned to $\text{sk}_{\text{Server}}^j$, the simulator finds the unique pair Client, i such that $\text{sid}_{\text{Client}}^i = \text{sid}_{\text{Server}}^j$ (that a unique such pair exists follows from a similar argument as in the previous proof) and sets $\text{sk}_{\text{Server}}^j = \text{sk}_{\text{Client}}^i$.

The modification described above is only made for noncorrupted Send_3 queries, and so in particular this means that the adversary did not make a query of the form $\text{Corrupt}_2(\text{Server}, \text{Client}, *)$ before initializing instance Π_{Server}^j (using a Send_1 query). It follows that instances $\Pi_{\text{Server}}^j, \Pi_{\text{Client}}^i$ use the same password. But in experiment P_3 , we always have $\text{sk}_{\text{Server}}^j = \text{sk}_{\text{Client}}^i$ whenever $\text{sid}_{\text{Server}}^j = \text{sid}_{\text{Client}}^i$ and instances $\Pi_{\text{Server}}^j, \Pi_{\text{Client}}^i$ use the same password. Thus $\text{fsAdv}_{\mathcal{A}, P_4}(k) = \text{fsAdv}_{\mathcal{A}, P_3}(k)$.

Experiment P_5 . Here we again modify the simulator's response to a noncorrupted Send_3 query. Upon receiving noncorrupted query $\text{Send}_3(\text{Server}, j, \langle K | \text{Sig} \rangle)$, the simulator examines $\text{msg}_1 = \text{first-msg-in}$ for instance Π_{Server}^j . If msg_1 is adversarially-generated and invalid, the session key $\text{sk}_{\text{Server}}^j$ is assigned a value randomly chosen from $\mathbb{G}$. In all other cases, the query is answered as in experiment P_4 . Exactly as in Claim 3.6, it is the case that $\text{fsAdv}_{\mathcal{A}, P_5}(k) = \text{fsAdv}_{\mathcal{A}, P_4}(k)$ (note in particular that the proof of Claim 3.6 holds even if the adversary learns the client's password at some later point).

Experiment P_6 . In experiment P_6 , we modify the simulator's response to noncorrupted Send_1 queries. Now, I is computed as $I := h^r g_1^{N+1}$; again, note that g_1^{N+1} is not a valid password. As in the proof of Claim 3.7, security of the (modified) Cramer-Shoup encryption scheme implies that $|\text{fsAdv}_{\mathcal{A}, P_5}(k) - \text{fsAdv}_{\mathcal{A}, P_6}(k)|$ is negligible under the DDH assumption.

Experiment P_7 . We modify the simulator's response to noncorrupted Send_2 queries. In response to a noncorrupted query $\text{Send}_2(\text{Client}, i, \text{msg}_2)$, the session key $\text{sk}_{\text{Client}}^i$ is assigned a value chosen randomly from $\mathbb{G}$ if either: (1) msg_2 is adversarially-generated and invalid; or (2) msg_2 is oracle-generated (recall that a msg_2 output in response to a *corrupted* Send_1 oracle query is *not* considered oracle-generated). In any other case, the session key is assigned a value as in experiment P_7 . As in the proof of Claim 3.8, we have $\text{fsAdv}_{\mathcal{A}, P_7}(k) = \text{fsAdv}_{\mathcal{A}, P_6}(k)$.

Experiment P_8 . In experiment P_8 , we modify the simulator's response to non-corrupted Send_0 queries. Now, C is computed as $C := h^r g_1^{N+1}$. As in the proof of Claim 3.9, security of the (modified) Cramer-Shoup encryption scheme implies that $|\text{fsAdv}_{\mathcal{A}, P_8}(k) - \text{fsAdv}_{\mathcal{A}, P_7}(k)|$ is negligible.

In experiment P_8 , the adversary can succeed in one of two ways: either it correctly guesses the value of the bit used by the Test oracle, or it succeeds as specified in experiment P_3 (namely, if it queries $\text{Test}(U, i)$ or $\text{Reveal}(U, i)$ with $\text{sk}_U^i = \nabla$, or if it queries $\text{Corrupt}_1(C)$ and there exists any instance associated with C which has session key ∇). The probability that it succeeds in the second manner is at most $Q(k)/N$, where $Q(k)$ is the number of on-line attacks made by $\mathcal{A}$ (cf. the definition of on-line attacks in Section 2.3). Otherwise, if the adversary queries $\text{Test}(U, i)$ for a fresh instance Π_U^i such that $\text{acc}_U^i = \text{TRUE}$, then sk_U^i is uniformly distributed in $\mathbb{G}$ independent of the adversary's view (since, in particular, Π_U^i is fresh only if the adversary's Send queries for this instance are noncorrupted). As in the previous proof, it follows that $\text{fsAdv}_{\mathcal{A}, P_8}(k) \leq Q(k)/N$ and so

$$\text{fsAdv}_{\mathcal{A}, P_0}(k) \leq Q(k)/N + \varepsilon(k)$$

for some negligible function $\varepsilon(\cdot)$. This concludes the proof of the theorem. $\square$

REFERENCES

- BELLARE, M., CANETTI, R., AND KRAWCZYK, H. 1998. A modular approach to the design and analysis of authentication and key exchange protocols. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, New York, 419–428.
- BELLARE, M., POINTCHEVAL, D., AND ROGAWAY, P. 2000. Authenticated key exchange secure against dictionary attacks. In *Proceedings of the Advances in Cryptology (Eurocrypt'00)*. Lecture Notes in Computer Science, vol. 1807. Springer-Verlag, Berlin, Germany, 139–155.
- BELLARE, M., AND ROGAWAY, P. 1993. Random oracles are practical: A paradigm for designing efficient protocols. *Proceedings of the 1st ACM Conference on Computer and Communications Security*, V. Ashby, Ed. ACM, New York, 62–73.
- BELLARE, M., AND ROGAWAY, P. 1994. Entity authentication and key distribution. In *Proceedings of Advances in Cryptology (Crypto'93)*. Lecture Notes in Computer Science, vol. 773. Springer-Verlag, Berlin, Germany, 232–249.
- BELLARE, M., AND ROGAWAY, P. 1995. Provably secure session key distribution: The three party case. In *Proceedings of the 27th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, New York, 57–66.
- BELLOVIN, S. M., AND MERRITT, M. 1992. Encrypted key exchange: Password-based protocols secure against dictionary attacks. In *Proceedings of the IEEE Symposium on Security & Privacy*. IEEE Computer Society Press, Los Alamitos, CA, 72–84.
- BELLOVIN, S. M., AND MERRITT, M. 1993. Augmented encrypted key exchange: A password-based protocol secure against dictionary attacks and password file compromise. In *Proceedings of the 1st ACM Conference on Computer and Communications Security*. ACM, New York, 244–250.
- BIRD, R., GOPAL, I., HERZBERG, A., JANSON, P., KUTTEN, S., MOLVA, R., AND YUNG, M. 1993. Systematic design of two-party authentication protocols. *IEEE J. Select. Areas Comm.* 11, 5, 679–693.
- BOYARSKY, M. 1999. Public-key cryptography and password protocols: The multi-user case. In *Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS'99)*. ACM, New York, 63–72.
- BOYKO, V., MACKENZIE, P. D., AND PATEL, S. 2000. Provably secure password-authenticated key exchange using Diffie-Hellman. In *Proceedings of Advances in Cryptology (Eurocrypt'00)*. Lecture Notes in Computer Science, vol. 1807. Springer-Verlag, Berlin, Germany, 156–171.
- CANETTI, R., GOLDBREICH, O., AND HALEVI, S. 2004. The random oracle methodology, revisited. *J. ACM* 51, 4, 557–594.

- CANETTI, R., HALEVI, S., KATZ, J., LINDELL, Y., AND MACKENZIE, P. D. 2005. Universally composable password-based key exchange. In *Proceedings of Advances in Cryptology (Eurocrypt'05)*. Lecture Notes in Computer Science, vol. 3494. Springer-Verlag, Berlin, Germany, 404–421.
- CANETTI, R., AND KRAWCZYK, H. 2001. Analysis of key-exchange protocols and their use for building secure channels. In *Proceedings of Advances in Cryptology (Eurocrypt'01)*. Lecture Notes in Computer Science, vol. 2045. Springer-Verlag, Berlin, Germany, 453–474.
- CANETTI, R., AND KRAWCZYK, H. 2002. Universally composable notions of key exchange and secure channels. In *Proceedings of Advances in Cryptology (Eurocrypt'02)*. Lecture Notes in Computer Science, vol. 2332. Springer-Verlag, Berlin, Germany, 337–351.
- CRAMER, R., AND SHOUP, V. 2003. Design and analysis of practical public-key encryption schemes secure against adaptive chosen ciphertext attack. *SIAM J. Comput.* 33, 1, 167–226.
- DAMGÅRD, I. 1988. Collision free hash functions and public key signature schemes. In *Proceedings of Advances in Cryptology (Eurocrypt'87)*. Lecture Notes in Computer Science, vol. 304. Springer-Verlag, New York, 203–216.
- DI CRESCENZO, G., KATZ, J., OSTROVSKY, R., AND SMITH, A. 2001. Efficient and non-interactive non-malleable commitment. In *Proceedings of the Advances in Cryptology (Eurocrypt'01)*. Lecture Notes in Computer Science, vol. 2045. Springer-Verlag, New York, 40–59.
- DI RAIMONDO, M., AND GENNARO, R. 2003. Provably secure threshold password-authenticated key exchange. In *Proceedings of the Advances in Cryptology (Eurocrypt'03)*. Lecture Notes in Computer Science, vol. 2656. Springer-Verlag, New York, 507–523.
- DIFFIE, W., AND HELLMAN, M. E. 1976. New directions in cryptography. *IEEE Trans. Inf. Theory* 22, 6, 644–654.
- DIFFIE, W., VAN OORSCHOT, P. C., AND WIENER, M. J. 1992. Authentication and authenticated key exchanges. *Designs, Codes Crypt.* 2, 2, 107–125.
- DOLEV, D., DWORK, C., AND NAOR, M. 2000. Nonmalleable cryptography. *SIAM J. Comput.* 30, 2, 391–437.
- EVEN, S., GOLDBREICH, O., AND MICALI, S. 1996. On-line/off-line digital signatures. *J. Crypt.* 9, 1, 35–67.
- GENNARO, R. 2008. Faster and shorter password-authenticated key exchange. In *Proceedings of the 5th Theory of Cryptography Conference (TCC'08)*. Lecture Notes in Computer Science, vol. 4948. Springer-Verlag, Berlin, Germany, 589–606.
- GENNARO, R., AND LINDELL, Y. 2006. A framework for password-based authenticated key exchange. *ACM Trans. Inf. Syst. Secur.* 9, 2, 181–234.
- GOLDBREICH, O., AND LINDELL, Y. 2006. Session-key generation using human passwords only. *J. Crypt.* 19, 3, 241–340.
- GOLDWASSER, S., MICALI, S., AND RIVEST, R. L. 1988. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM J. Comput.* 17, 2, 281–308.
- GONG, L. 1995. Optimal authentication protocols resistant to password guessing attacks. In *Proceedings of the 8th IEEE Computer Security Foundations Workshop*. IEEE Computer Society Press, Los Alamitos, CA, 24–29.
- GONG, L., LOMAS, T. M. A., NEEDHAM, R. M., AND SALTZER, J. H. 1993. Protecting poorly chosen secrets from guessing attacks. *IEEE J. Select. Areas Comm.* 11, 5, 648–656.
- HALEVI, S., AND KRAWCZYK, H. 1999. Public-key cryptography and password protocols. *ACM Trans. Inf. Syst. Secur.* 2, 3, 230–268.
- JABLON, D. P. 1997. Extended password key exchange protocols immune to dictionary attacks. In *Proceedings of the 6th IEEE International Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises*. IEEE Computer Society Press, Los Alamitos, CA, 248–255.
- JIANG, S., AND GONG, G. 2004. Password based key exchange with mutual authentication. In *Proceedings of the 11th Annual International Workshop on Selected Areas in Cryptography (SAC)*. Lecture Notes in Computer Science, vol. 3357. Springer-Verlag, Berlin, Germany, 267–279.
- KATZ, J. 2002. Efficient cryptographic protocols preventing “man-in-the-middle” attacks. Ph.D. dissertation, Columbia University, New York.
- KATZ, J., AND LINDELL, Y. 2007. *Introduction to Modern Cryptography*. Chapman & Hall/CRC Press.
- KATZ, J., MACKENZIE, P., TABAN, G., AND GLIGOR, V. 2005. Two-server password-only authenticated key exchange. In *Proceedings of the 3rd International Conference on Applied Cryptography and Network Security (ACNS)*. Lecture Notes in Computer Science, vol. 3531. Springer-Verlag, Berlin, Germany, 1–16.
- KATZ, J., OSTROVSKY, R., AND YUNG, M. 2001. Efficient password-authenticated key exchange using human-memorable passwords. In *Proceedings of the Advances in Cryptology (Eurocrypt'01)*. Lecture Notes in Computer Science, vol. 2045. Springer-Verlag, Berlin, Germany, 475–494.

- KATZ, J., OSTROVSKY, R., AND YUNG, M. 2002. Forward secrecy in password-only key exchange protocols. In *Proceedings of the 3rd International Conference on Security in Communication Networks (SCN)*. Lecture Notes in Computer Science, vol. 2576. Springer-Verlag, Berlin, Germany, 29–44.
- KLEIN, D. V. 1990. Foiling the cracker: A survey of, and improvements to, password security. In *Proceedings of the 2nd USENIX Unix Security Workshop*. 5–14.
- LOMAS, T. M. A., GONG, L., SALTZER, J. H., AND NEEDHAM, R. M. 1989. Reducing risks from poorly chosen keys. In *Proceedings of the 12th ACM Symposium on Operating System Principles*. ACM, New York, 14–18.
- LUCKS, S. 1997. Open key exchange: How to defeat dictionary attacks without encrypting public keys. In *Security Protocols Workshop*. Lecture Notes in Computer Science, vol. 1361. Springer-Verlag, Berlin, Germany, 79–90.
- MACKENZIE, P., PATEL, S., AND SWAMINATHAN, R. 2000. Password-authenticated key exchange based on RSA. In *Proceedings of Advances in Cryptology (Asiacrypt'00)*. Lecture Notes in Computer Science, vol. 1976. Springer-Verlag, Berlin, Germany, 599–613.
- MENEZES, A. J., VAN OORSCHOT, P. C., AND VANSTONE, S. A. 1996. *Handbook of Applied Cryptography*. CRC Press.
- MORRIS, R., AND THOMPSON, K. 1979. Password security—a case history. *Comm. ACM* 22, 11, 594–597.
- NGUYEN, M.-H., AND VADHAN, S. 2008. Simpler session-key generation from short random passwords. *J. Crypt.* 21, 1, 52–96.
- PATEL, S. 1997. Number theoretic attacks on secure password schemes. In *Proceedings of the IEEE Symposium on Security & Privacy*. IEEE Computer Society Press, Los Alamitos, CA, 236–247.
- SHOUP, V. 1999. On formal models for secure key exchange. Tech. rep. RZ 3120, IBM. <http://eprint.iacr.org/1999/012>.
- SHOUP, V. 2001. A proposal for an ISO standard for public key encryption. <http://eprint.iacr.org/2001/112>.
- SPAFFORD, E. 1992. Observing reusable password choices. In *Proceedings of the 3rd USENIX Security Symposium*. USENIX Association, 299–312.
- STEINER, M., TSUDIK, G., AND WAIDNER, M. 1995. Refinement and extension of encrypted key exchange. *ACM SIGOPS Operat. Syst. Rev.* 29, 3, 22–30.
- WU, T. D. 1998. The secure remote password protocol. In *Proceedings of the Network and Distributed System Security Symposium (NDSS'98)*. The Internet Society, 97–111.
- WU, T. D. 1999. A real-world analysis of Kerberos password security. In *Proceedings of the Network and Distributed System Security Symposium (NDSS'99)*. The Internet Society, 13–22.

RECEIVED AUGUST 2004; REVISED APRIL 2007; ACCEPTED JULY 2009